



第六章

传输层

陆有军 tjlyj@tongji.edu.cn



传输层

✍ 传输服务

✍ 传输协议的要素

✍ 拥塞控制

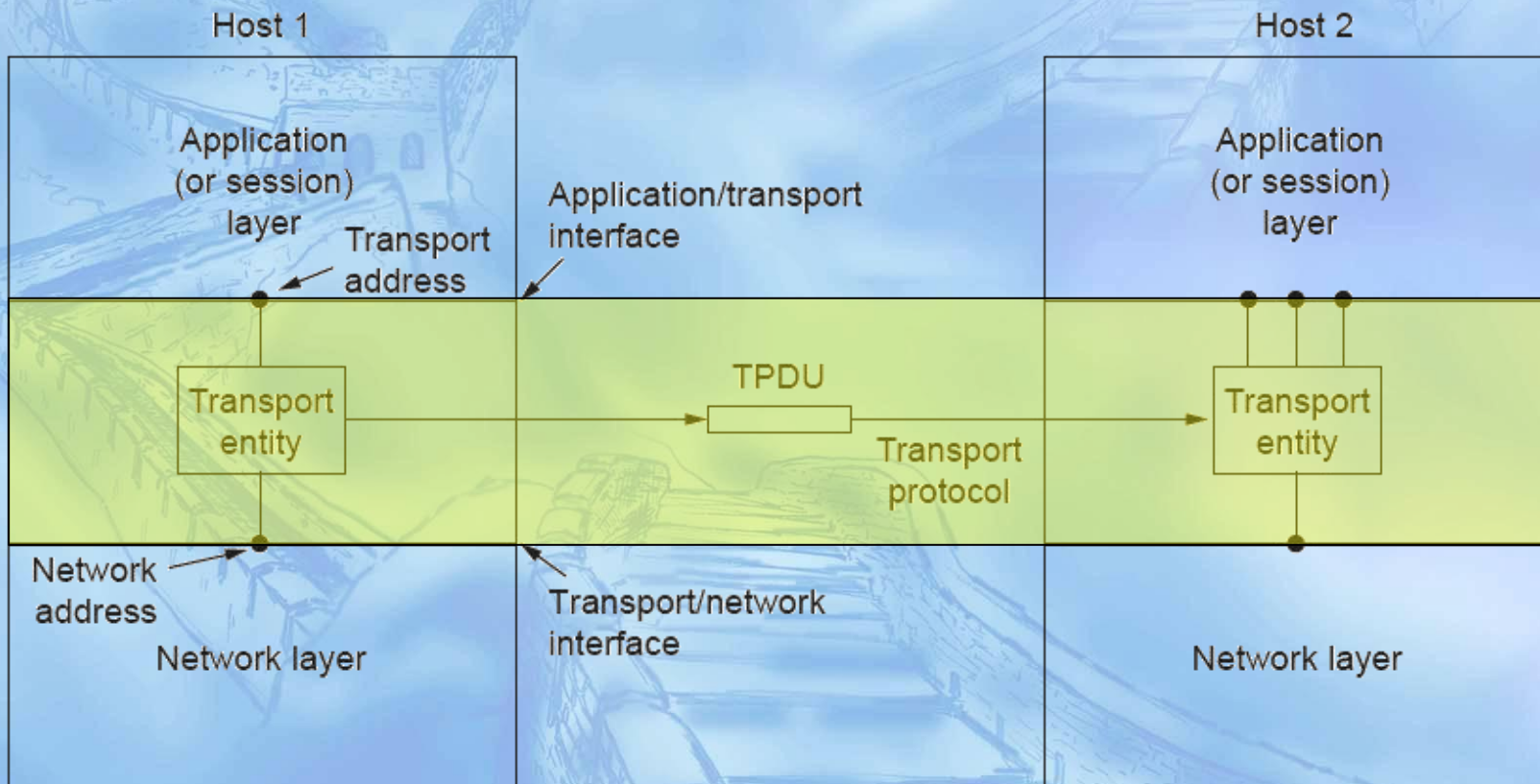
✍ Internet传输协议:UDP

✍ Internet传输协议:TCP

✍ 性能问题



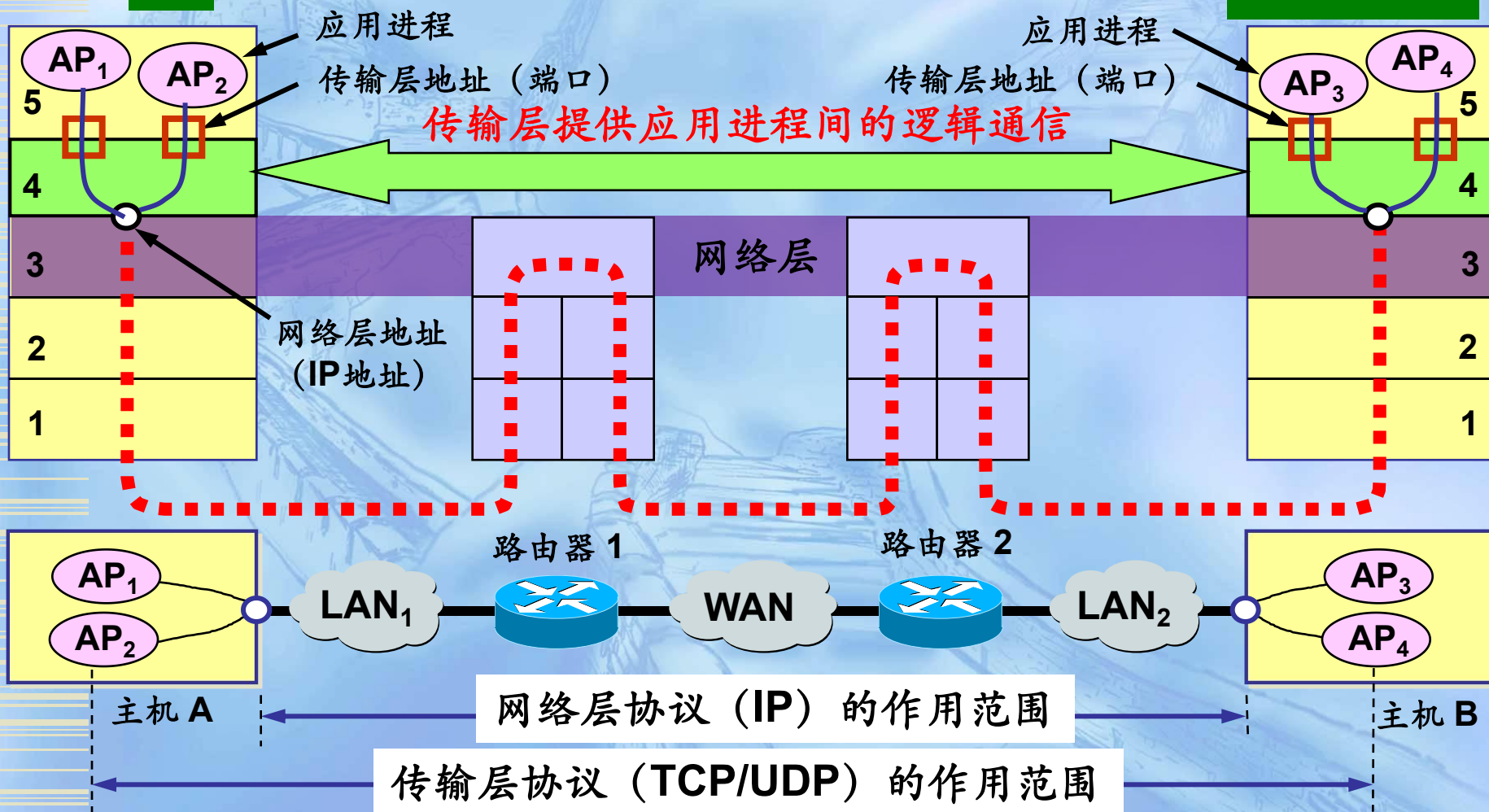
传输层



传输实体可在OS内核实现，或者以一个链接库形式绑定到网络应用中，或者以一个独立的用户进程运行，甚至可以实现在NIC上。



例:Internet中的传输层





提供给上层的服务

✍ 目的

向它的用户（通常是应用层的进程）提供高效的、可靠的和成本有效的数据传输服务。

✍ 服务类型

✦ 面向连接的传输服务

与面向连接的网络服务类似。例如：Internet中的TCP协议；

✦ 无连接的传输服务

与无连接的网络服务类似。例如：Internet中的UDP协议。



疑问？

✍ 问题：

既然传输层服务与网络层服务如此类似，为什么还要设置两个独立的层？

✍ 答案：

- ✦ 传输层代码完全运行在用户的机器上，而网络层代码主要运行在由运营商控制的路由器上，用户对网络层没有真正的控制权，所以需要传输层来提高网络的服务质量；
- ✦ 传输服务原语可以通过调用库程序来实现，从而使这些原语独立于网络服务原语。



传输服务原语

Primitive	Packet sent	Meaning
LISTEN	(none)	Block until some process tries to connect
CONNECT	CONNECTION REQ.	Actively attempt to establish a connection
SEND	DATA	Send information
RECEIVE	(none)	Block until a DATA packet arrives
DISCONNECT	DISCONNECTION REQ.	This side wants to release the connection

✍ 一个简单的面向连接的传输服务

✦ 建立连接：**LISTEN、CONNECT**原语

✦ 数据传输：**SEND、RECEIVE**原语

✦ 释放连接：**DISCONNECT**原语

⊕ 非对称释放：任何一方发**DISCONNECT**原语

⊕ 对称释放：双方都需发**DISCONNECT**原语

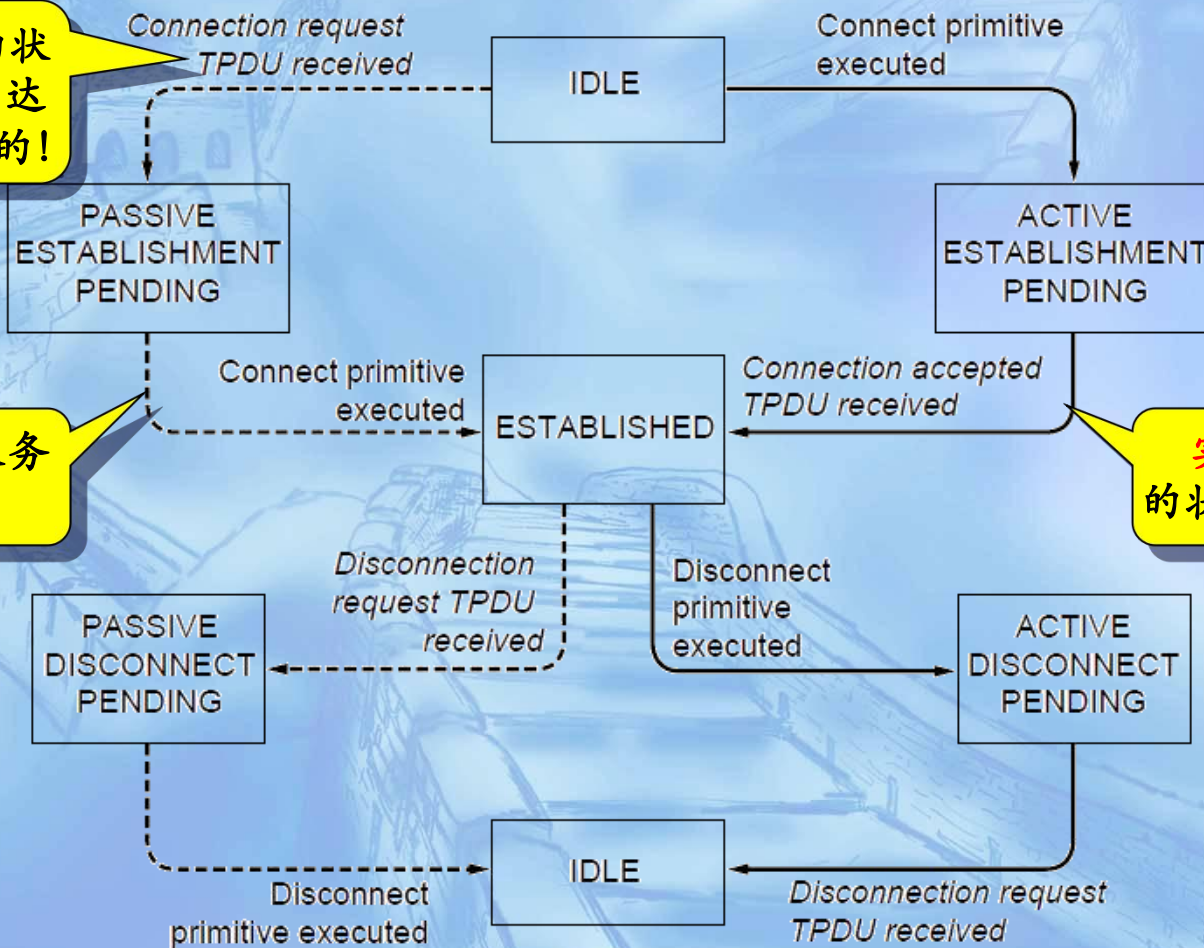


建立和释放连接

斜体标记的状态转移是由到达的TPDU引起的!

虚线表示服务器的状态序列

实线表示客户的状态序列





传输协议的要素

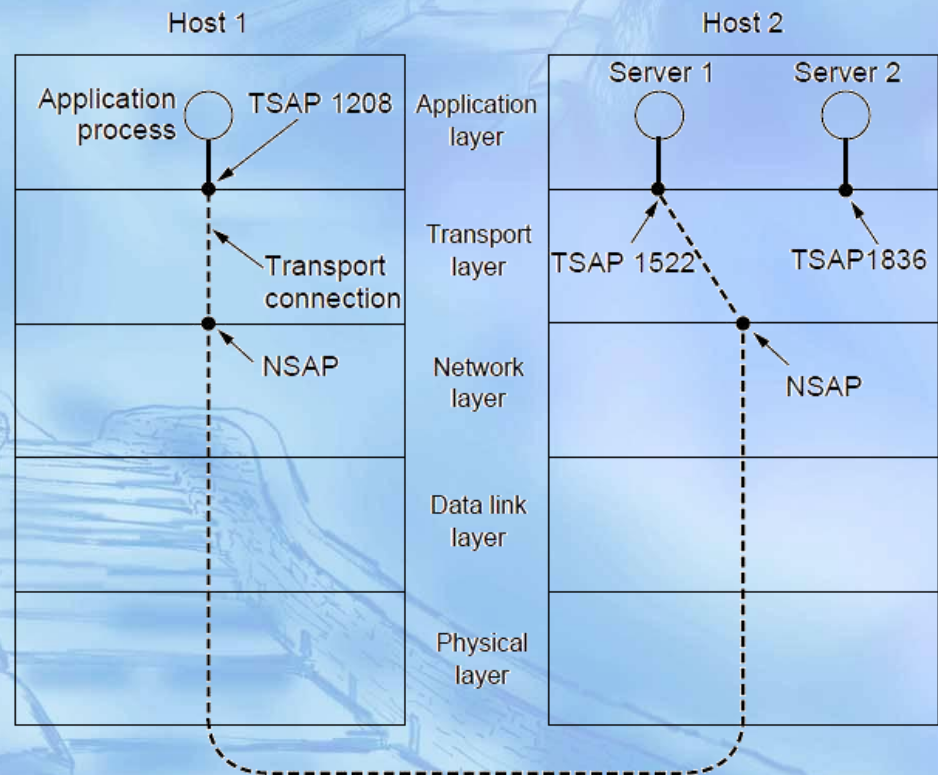
-  寻址
-  连接建立
-  连接释放
-  差错控制和流量控制
-  多路复用
-  崩溃恢复



传输服务访问点 (TSAP)

✍ **TSAP(传输服务访问点)**: 传输层的地址, 用以区分应用层的不同进程。例如, Internet 中的端口。

✍ **NSAP(网络服务访问点)**: 网络层的地址, 用以区分网络中的不同计算机。例如, Internet 中的 IP 地址。





寻 址

✂ **问题：**如何知道通信对方的**TSAP**地址？

✂ **解决：**

✦ **方法1：固定的TSAP地址**

- ✦ **思想：**将服务器进程与固定的**TSAP**地址进行绑定，并广而告之。
- ✦ **适用：**适用于少数永不改变的关键服务，例如，**Web**服务器。
- ✦ **问题：**如果通信对方是用户进程，因为用户进程的**TSAP**地址无法预知，且只存在较短时间，如何？



寻 址

✂ **问题：**如何知道通信对方的**TSAP**地址？

✂ **解决：**

✦ **方法2：端口映射器**

- ✦ 端口映射器是一个特殊的进程，内有数据库，保存各种服务名字与其对应**TSAP**地址的对应关系，提供查询服务，使用知名的**TSAP**地址；
- ✦ 用户如需查询一个特定服务（例：**BitTorrent**）相对应的**TSAP**地址，可与端口映射器通信进行查询；
- ✦ 当一个新服务被创建时必须向端口映射器注册。



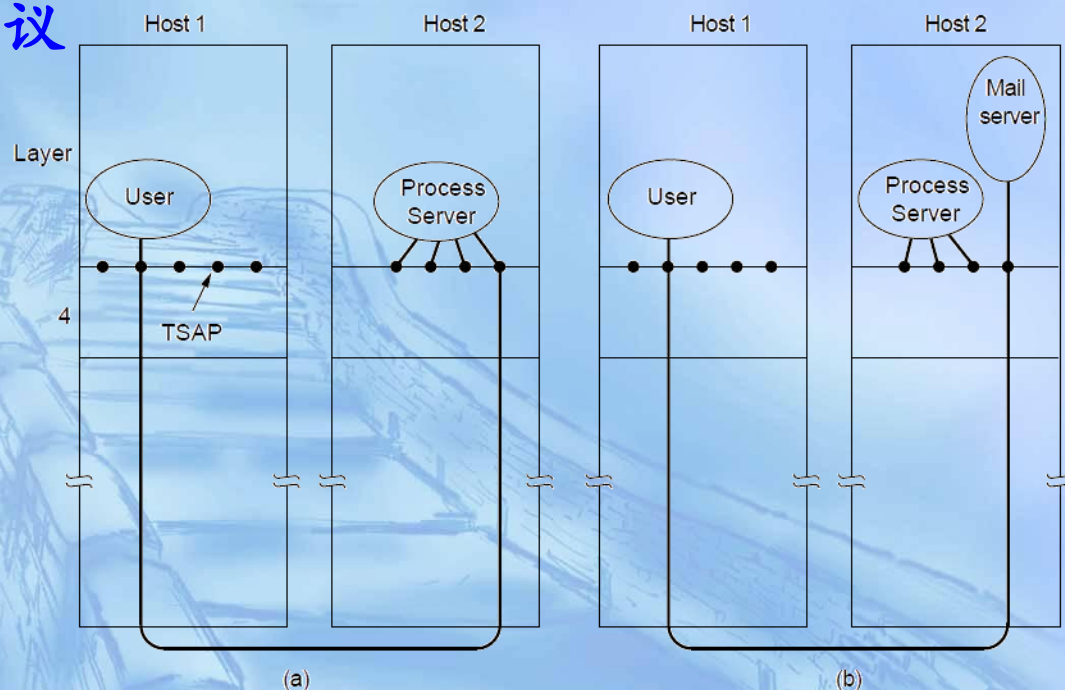
初始连接协议

✂ **问题：**1台运行多个（不常使用）网络服务的计算机，如果让每个服务器进程保持活跃，并整天监听一个稳定的TSAP地址，则是一种浪费？

✂ **解决：**初始连接协议

进程服务器是一个特殊的进程，充当那些不常使用的服务器的代理。

本方法只适用于服务器可按需创建的场所。





连接建立

✍ **问题：**当网络出现丢失、延迟、损坏或重复数据包时，如何确保连接建立的可靠性？

✍ **解决：**

- ✍ 对于数据包的丢失或损坏可以采用**ARQ**和**校验和**来解决；
- ✍ 对于数据包的延迟或重复，不能将一个旧的、重复的数据包当作新的数据包处理，重点介绍。



延迟重复的处理

✎ 限制数据包（网络层）的生存期

✦ 限制网络设计

✦ 在每个数据包中放置一个跳计数器

✦ 为每个数据包打上时间戳

✎ 每个段（**传输层**）携带不同**序号**，在**周期 $T=2 \times \text{MSL}$** （最大数据包生存期，例：**Internet**取**120秒**）时间内序号空间不会重复，使用**日时钟**（二进制计数器）防止计算机崩溃丢失序号

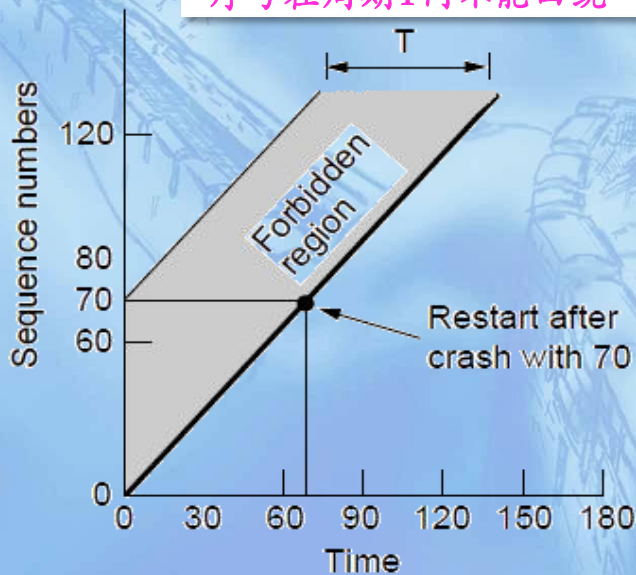
✎ **三次握手**建立连接：确定初始序号



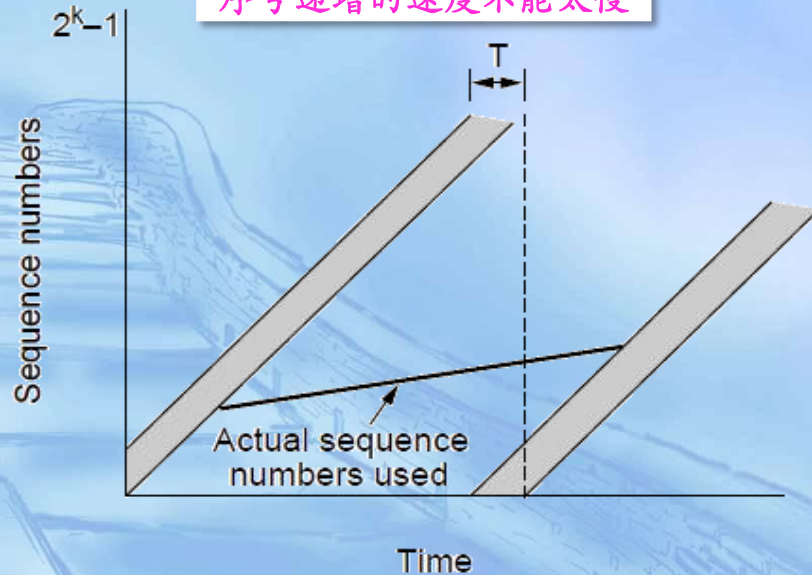
日时钟

- ✍ **日时钟**以统一时间间隔递增，位数 $\geq$ 序号位数，不停止
- ✍ **序号空间应该足够大**，在**周期T**内序号不会回绕（下图左）：对于时钟速率为**C**，序号空间大小为**S**，则必须满足 **$S/C > T$**

序号在周期T内不能回绕



序号递增的速度不能太慢

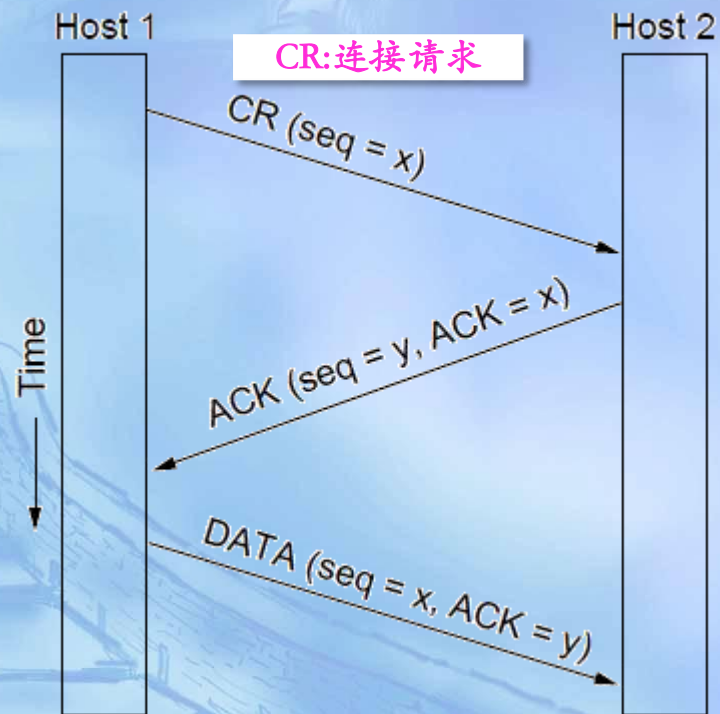




三次握手

✍ **三次握手用于连接建立。**因为通常不会记住对方在该连接上的序号，所以无法判断对方的初始序号是否重复

- ✦ 主机1发起连接请求，初始序号为 x
- ✦ 主机2回应一个ACK段作为对 x 的确认，并告知它的初始序号为 y
- ✦ 主机1在第一个数据段中对序号 y 进行确认

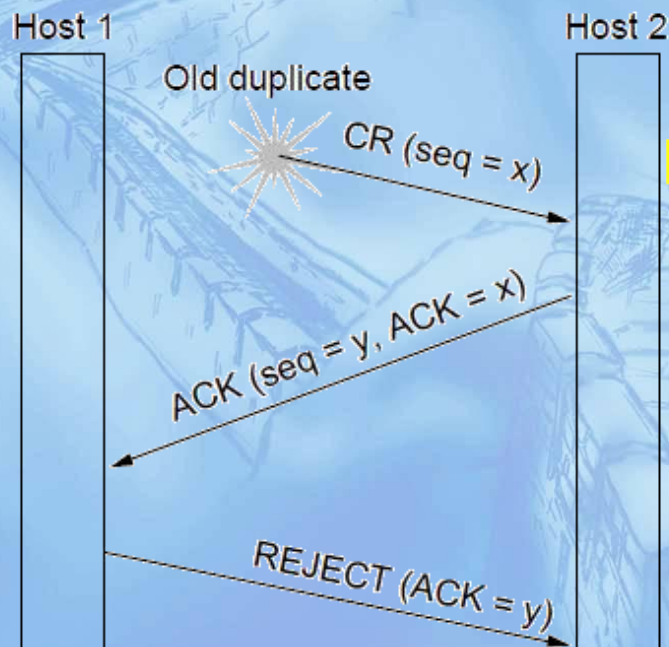




三次握手

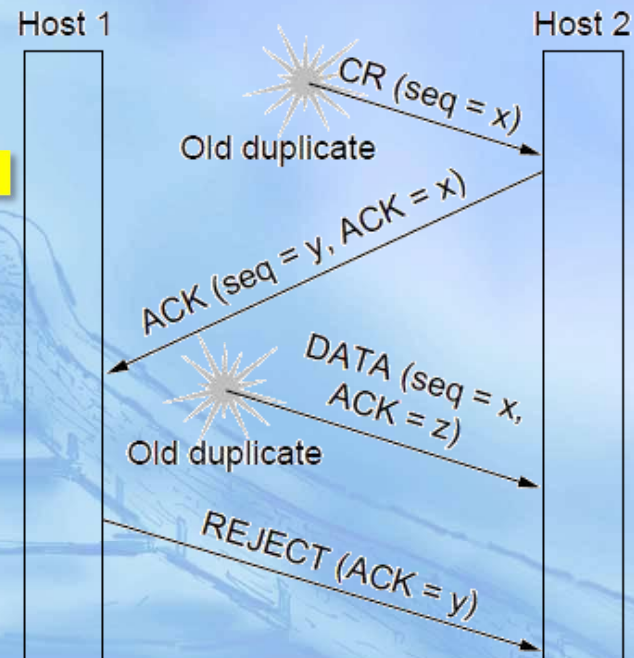
✂ 三次握手协议能**正确**处理延迟的、重复的控制段

出现旧的、重复的连接请求



CR:连接请求

出现重复的连接请求和重复的ACK





例:三次握手

✍ **TCP**协议使用三次握手来建立连接

- ✦ 序号字段为**32bit**
- ✦ **PAWS**（防止序号回绕）：在连接期间，时间戳被用来辅助扩展**32bit**序号，以便它在周期T内不会回绕，甚至对于每秒千兆位的连接也是一样
- ✦ **伪随机的初始序号**：防止攻击者预测到下一个初始序号，但注意伪随机的初始序号在一个时间间隔内不能重复



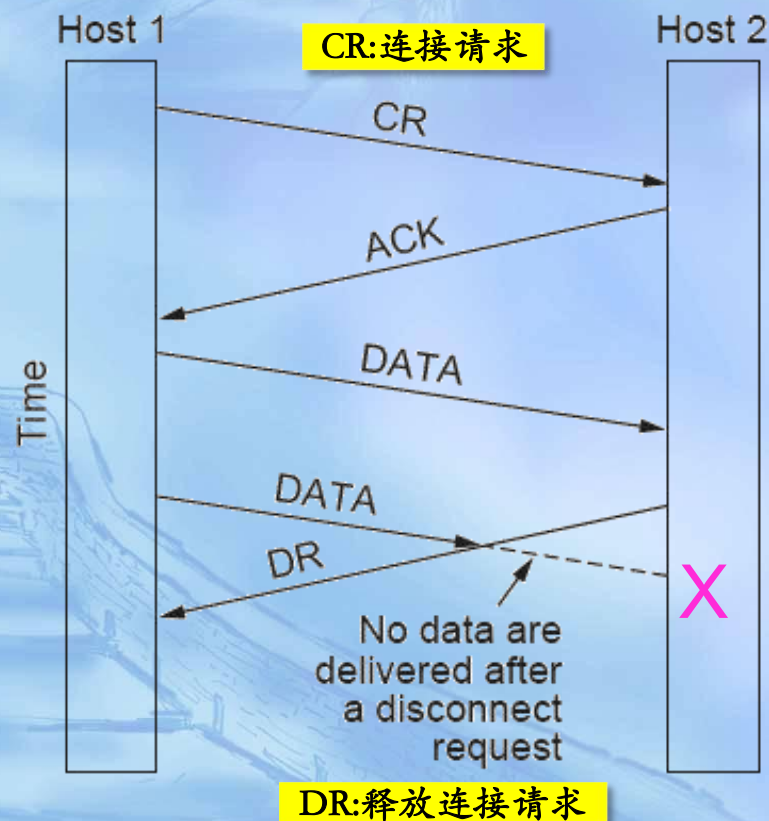
连接释放

✎ **问题：**在连接释放时如何确保可靠性？

✎ **解决：**

✦ **非对称释放**（右图）：
由任何一方释放连接，
本方法太冒失，可能会
丢失数据（✗）

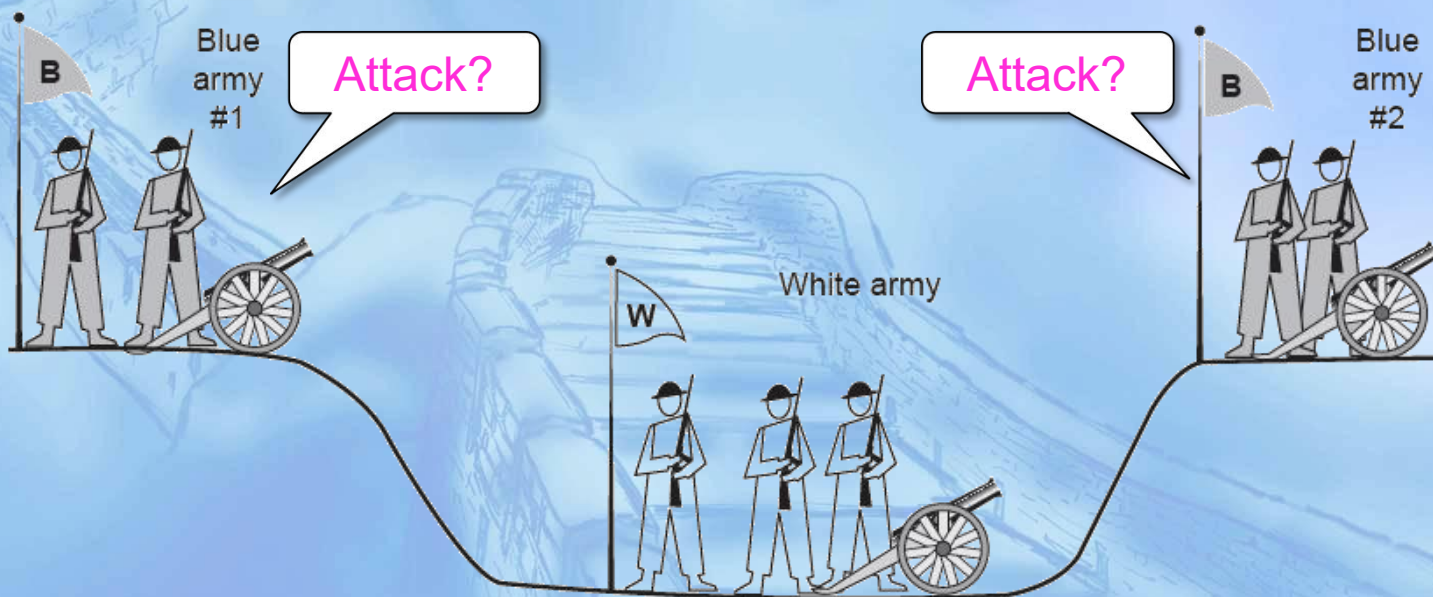
✦ **对称释放**：将连接看成
两个独立的单向连接，
要求单独释放每一个单
向连接（✓）





对称释放连接

- ✍ **对称释放**（双方都同意释放）不能完全由传输层处理
 - ✚ 两军问题显示出该协议中的陷阱



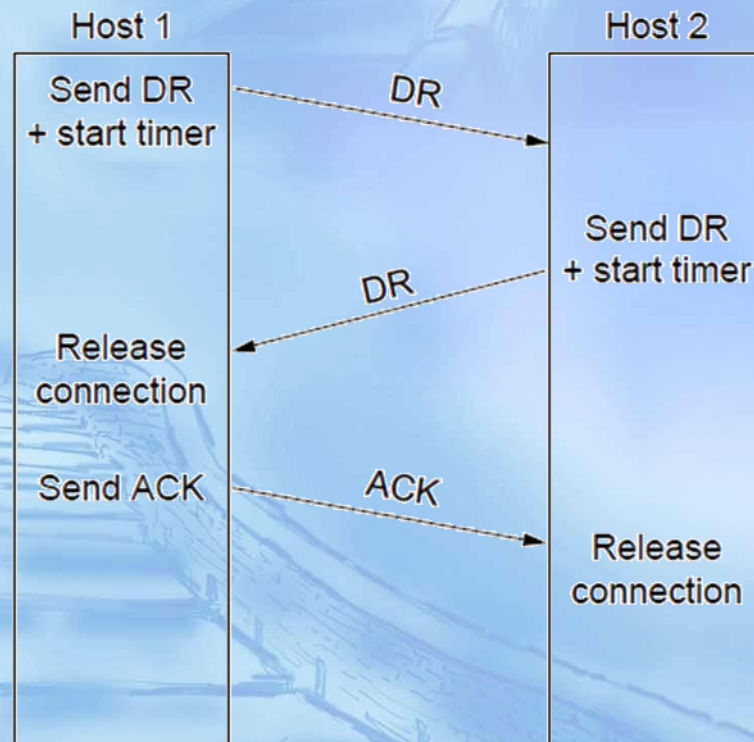


对称释放连接

✍ **对称释放**：由连接两端的**传输层用户**独立决定是否释放连接

✦ 正常情况

- ✦ 释放连接最初是由主机1上的传输层用户发起的
- ✦ 两个DR都是由另一端进行确认的



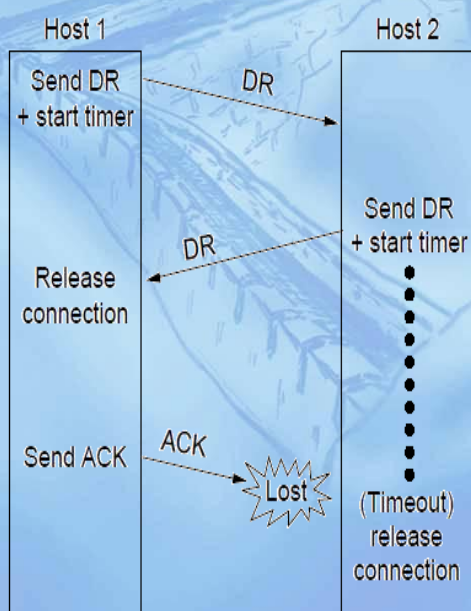
DR:释放连接请求



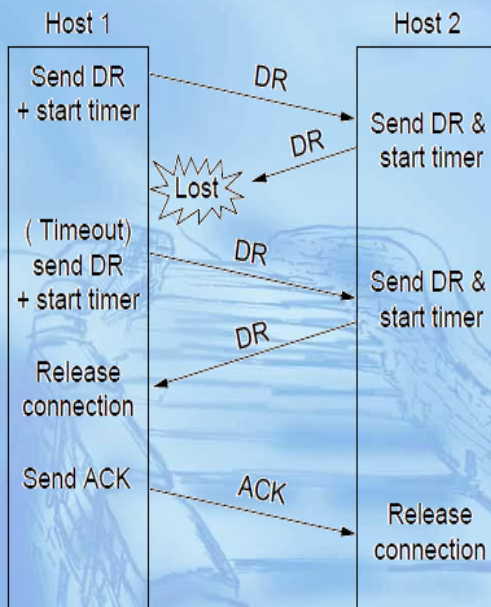
对称释放连接

✍ **对称释放**：由连接两端的**传输层用户**独立决定是否释放连接

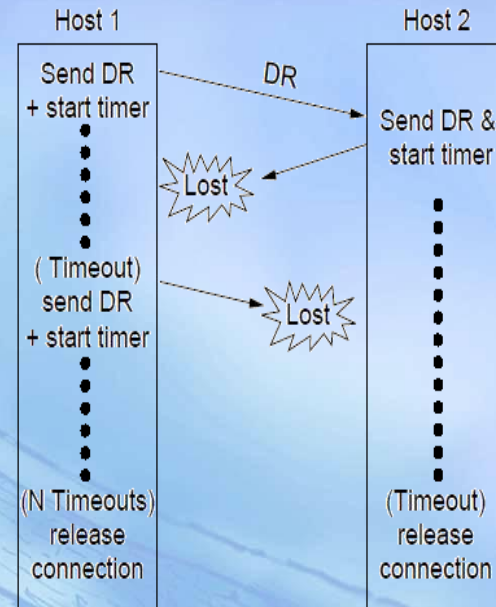
✚ **异常情况**：错误由**计时器**和**重传**解决



最后的ACK段丢失，主机2超时，释放连接



第2个DR丢失导致主机1重传，再次尝试释放连接



极端:很多DRs丢失导致两端主机超时，并强制释放连接



差错控制和流量控制

✍ 差错控制

- ✚ 目的：确保数据传输的可靠性
- ✚ 方法：带校验和和重传机制的滑动窗口协议（数据链路层）

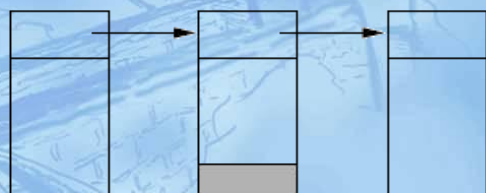
✍ 流量控制

- ✚ 目的：防止快速发送端淹没慢速接收端
- ✚ 方法：管理发送端/接收端的缓冲区
 - ✚ 数据在不同的时间去/来自网络和应用程序
 - ✚ 窗口告诉发送端：接收端的可用缓冲区
 - ✚ 动态滑动窗口协议
 - ✚ 发送端和接收端需要动态调整缓冲区的分配



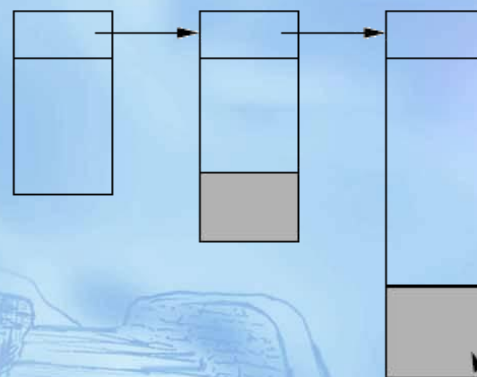
缓冲区的组织

✎ 不同的缓冲策略：效率和复杂性



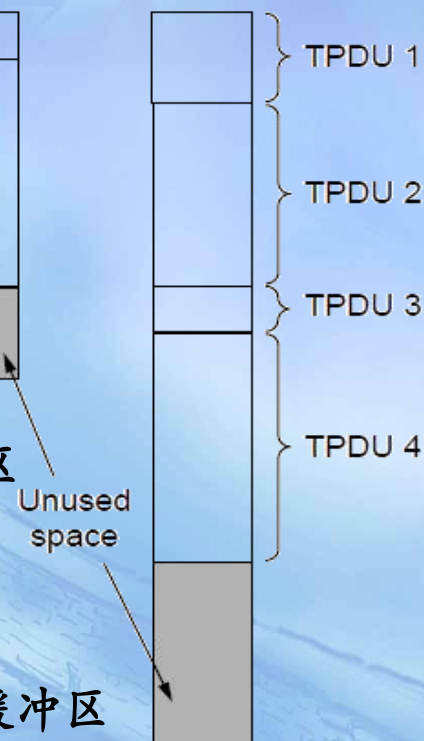
链式固定大小的缓冲区

(a)



(b)

链式可变大小的缓冲区



c) 一个大的循环缓冲区



动态缓冲区分配

A	Message	传输方向	B	B's Buffer	Comments
1	→ < request 8 buffers >	→			A wants 8 buffers
2	← < ack = 15, buf = 4 >	←		0 1 2 3	B grants messages 0-3 only
3	→ < seq = 0, data = m0 >	→		0 1 2 3	A has 3 buffers left now
4	→ < seq = 1, data = m1 >	→		0 1 2 3	A has 2 buffers left now
5	→ < seq = 2, data = m2 >	→	...	0 1 2 3	Message lost but A thinks it has 1 left
6	← < ack = 1, buf = 3 >	←		1 2 3 4	B acknowledges 0 and 1, permits 2-4
7	→ < seq = 3, data = m3 >	→		1 2 3 4	A has 1 buffer left
8	→ < seq = 4, data = m4 >	→		1 2 3 4	A has 0 buffers left, and must stop
9	→ < seq = 2, data = m2 >	→		1 2 3 4	A times out and retransmits
10	← < ack = 4, buf = 0 >	←		1 2 3 4	Everything acknowledged, but A still blocked
11	← < ack = 4, buf = 1 >	←		2 3 4 5	A may now send 5
12	← < ack = 4, buf = 2 >	←		3 4 5 6	B found a new buffer somewhere
13	→ < seq = 5, data = m5 >	→		3 4 5 6	A has 1 buffer left
14	→ < seq = 6, data = m6 >	→		3 4 5 6	A is now blocked again
15	← < ack = 6, buf = 0 >	←		3 4 5 6	A is still blocked
16	... < ack = 6, buf = 4 >	←		7 8 9 10	Potential deadlock

丢失

动态滑动窗口协议



动态滑动窗口协议

✍ 动态滑动窗口协议可以同时实现流量控制和拥塞控制

- ✚ 流量控制：发送端的窗口大小取决于接收端的可用缓冲区容量

- ✚ 拥塞控制

 - ✚ 发送端的窗口大小取决于网络的承载能力

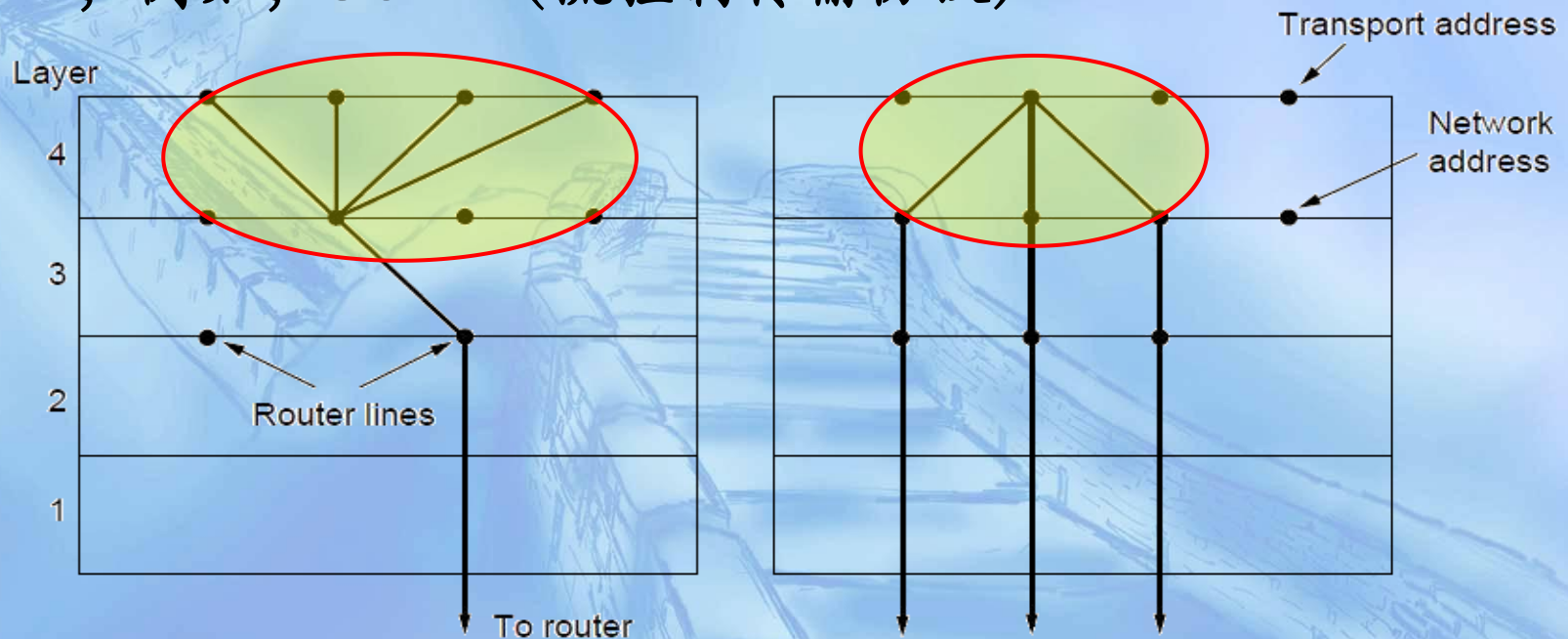
 - ✚ 例：若网络每秒种能够处理 c 段，且往返时间是 r ，则发送端的窗口大小应该为 cr

 - ✚ 发送端的窗口大小应该频繁地调整以适应当前网络承载能力的变化



多路复用

- ✍ **多路复用**：多个独立的传输连接使用同一个网络连接，例如，**TCP**
- ✍ **逆向多路复用**：一个传输连接使用多个独立的网络连接，例如，**SCTP**（流控制传输协议）





崩溃恢复

- ✍ **网络或路由器崩溃**：传输层通过**重传**进行恢复
- ✍ **主机崩溃**：需要**应用层**的帮助，因为确认和写入不是原子的，会导致传输层恢复失败（例子见下图）

Strategy used by receiving host

Strategy used by sending host	First ACK, then write			First write, then ACK		
	AC(W)	AWC	C(AW)	C(WA)	W AC	WC(A)
Always retransmit	OK	DUP	OK	OK	DUP	DUP
Never retransmit	LOST	OK	LOST	LOST	OK	OK
Retransmit in S0	OK	DUP	LOST	LOST	DUP	OK
Retransmit in S1	LOST	OK	OK	OK	OK	DUP

S0:没有未完成的段

S1:发出一个段，尚未确认

OK = Protocol functions correctly

DUP = Protocol generates a duplicate message

LOST = Protocol loses a message

A:发送一个确认

W:将数据写入进程

C:崩溃

同济大学.计算机科学与技术学院



拥塞控制

- ✍ **拥塞**: 如果许多计算机上的传输实体以太快的速度发送太多的数据包, 就会使得网络不堪重负而变得拥塞, 继而数据包被延迟和丢失, 从而导致网络性能严重下降
- ✍ **拥塞控制**: 由网络层和传输层共同负责
 - ✦ **网络层**: 拥塞发生在路由器上, 因此在网络层检测拥塞
 - ✦ **传输层**: 拥塞的根源是传输层注入到网络中的流量引起的, 因此控制拥塞的唯一途径是传输层放缓向网络中发送数据包的速度



拥塞控制

~~✍~~ 理想的带宽分配

~~✍~~ 调整发送速率

~~✍~~ 无线问题



理想的带宽分配

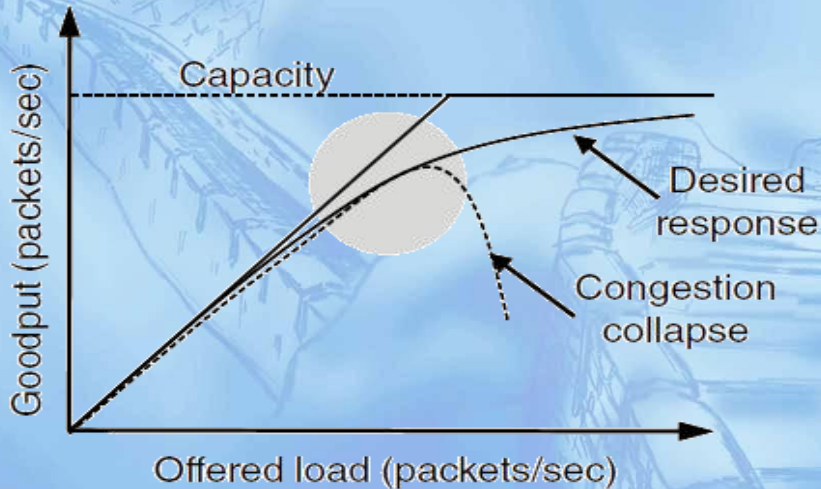
- ✍ 拥塞控制算法的目标是更加易于避免拥塞，即为使用网络的传输层找到一种好的**带宽分配**方法
- ✍ 有效的带宽分配能提供良好性能：
 - ✦ 能利用所有的可用带宽却能避免拥塞
 - ✦ 对于整个竞争的传输实体是公平的
 - ✦ 能快速跟踪流量需求的变化



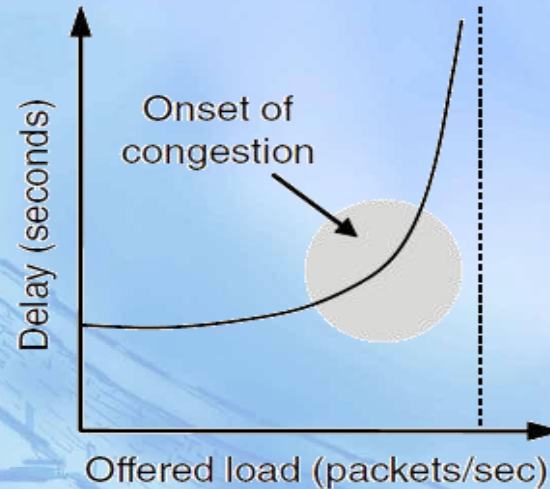
效率和功率

✎ 有效的带宽分配能提供高吞吐量、低时延

- ✎ 为表示最佳效率，引入**功率**度量：**功率=负载/延迟**
- ✎ 达到最大功率的负载表示了传输实体放置在网络上的有效负载



当拥塞开始出现时，吞吐量比负载更缓慢增长



当拥塞开始出现时，延迟开始急剧上升

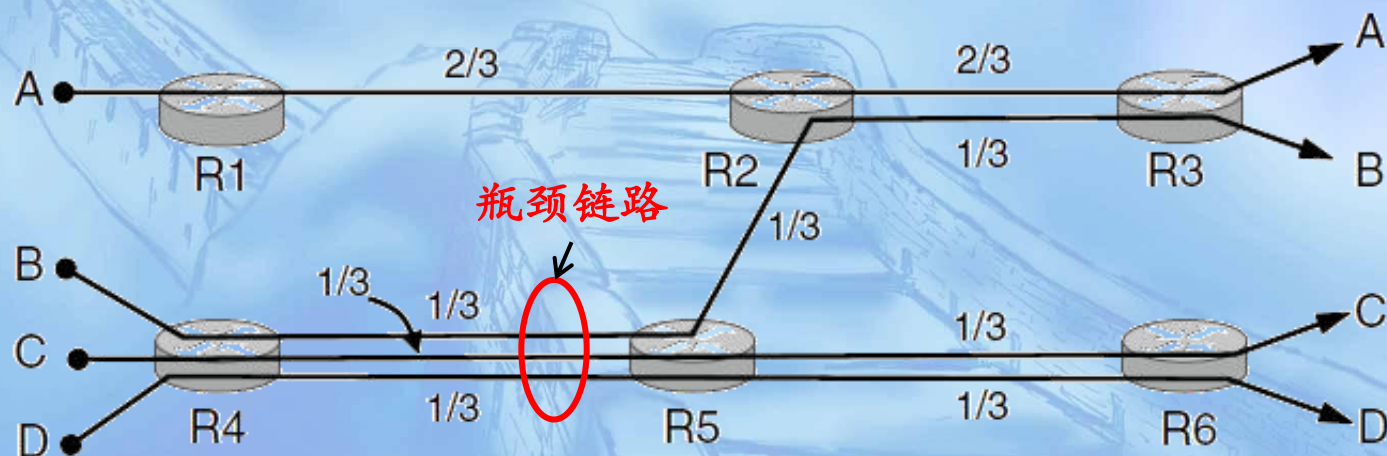


最大-最小公平

✎ **公平**是指给所有的流分配带宽（没有饥饿）

✦ **最大-最小公平**：如果分配给一个流的带宽在不减少分配给另一个流带宽的前提下无法得到进一步增长，那么就不给这个流分配更多带宽

✦ 最大-最小公平在瓶颈提供公平的共享

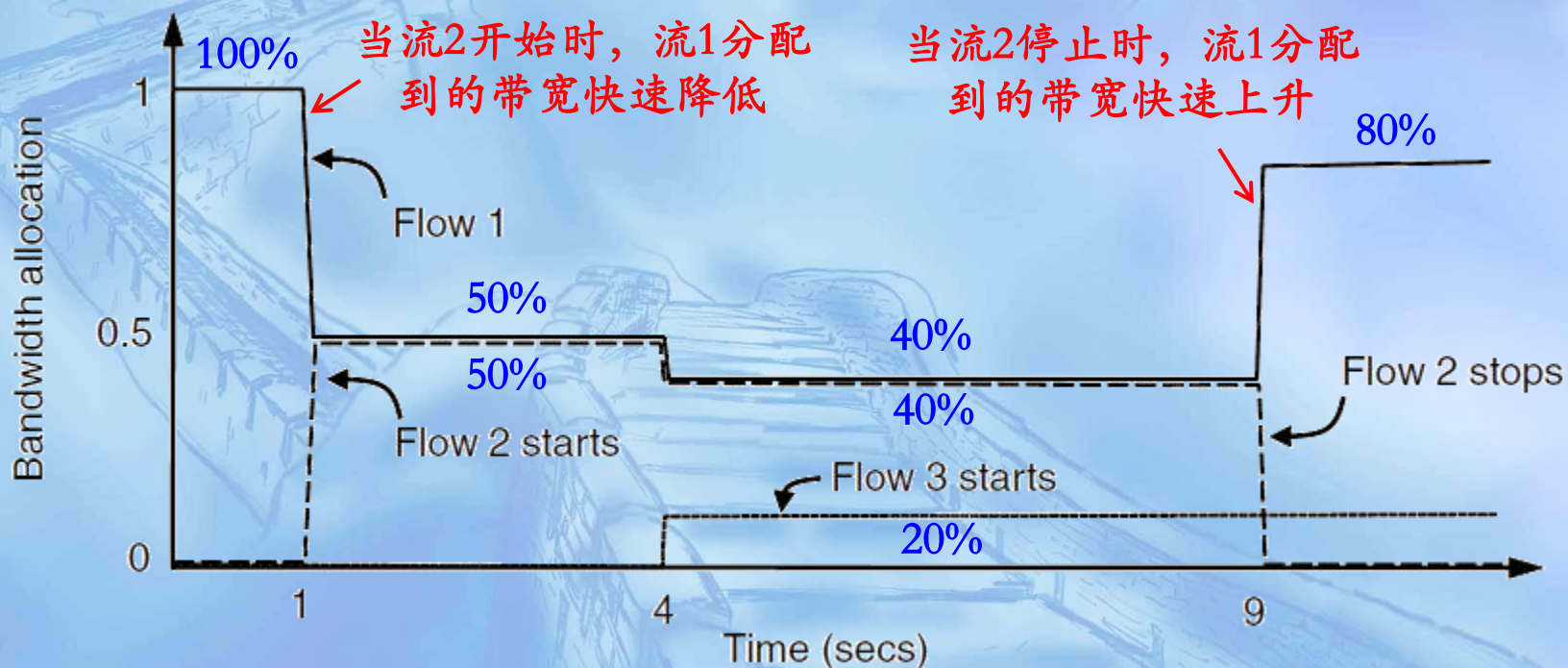


每条链路具有相同的容量，用1表示



收敛

- ✎ 一个良好的拥塞控制算法能够随着通信模式的变化快速收敛到公平而有效的带宽分配上。



随着时间的推移而变化的带宽分配

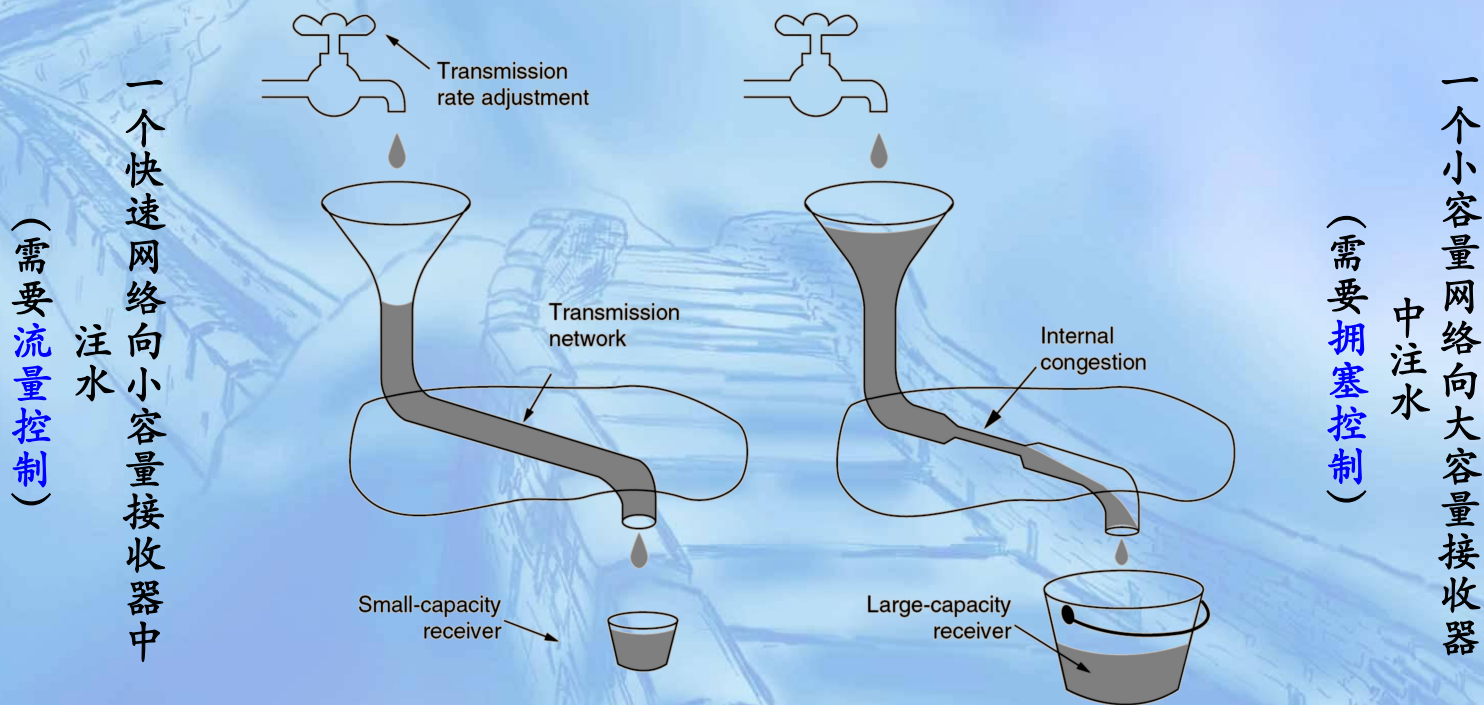


发送速率

✎ 发送速率受制于:

✎ **流量控制**: 接收端没有足够缓冲区 (左图)

✎ **拥塞控制**: 网络没有足够的容量 (右图)





网络拥塞信号

✎ 调节发送速率的方式依赖于网络拥塞信号的反馈形式

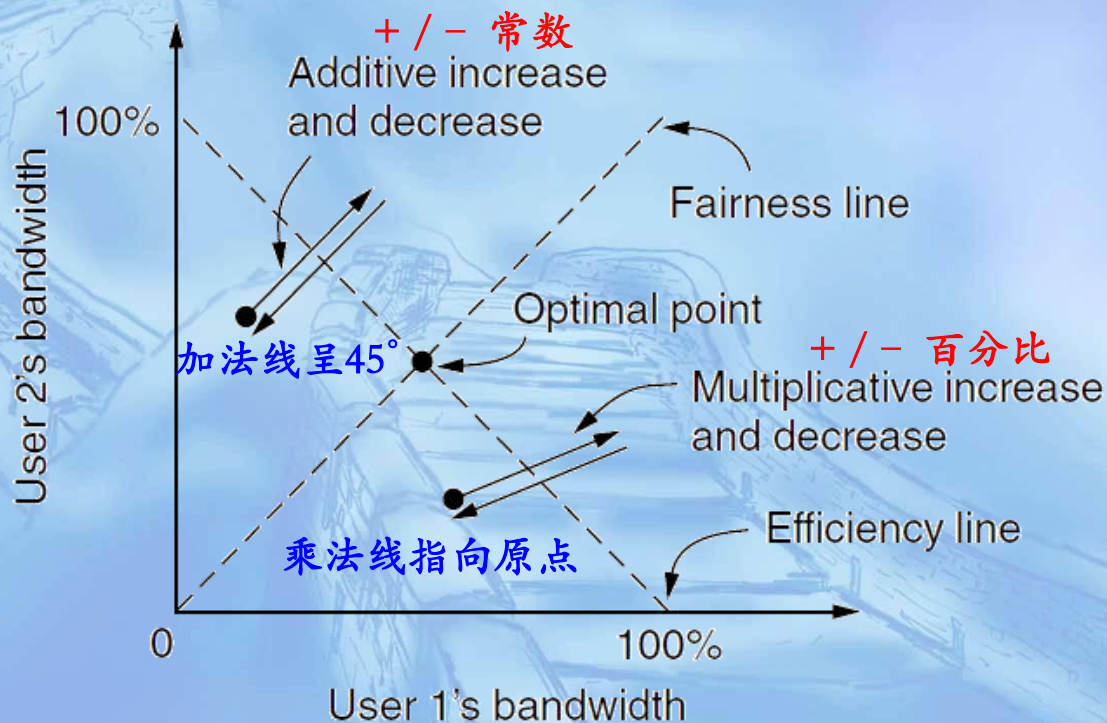
- ✎ 显式/隐式
- ✎ 精确/不精确

协议	信号	显式否	精确否
XCP	使用速率	是	是
TCP with ECN	拥塞警告	是	否
FAST TCP	端到端延迟	否	是
Compound TCP	数据包丢失和端到端延迟	否	是
CUBIC TCP	数据包丢失	否	否
TCP	数据包丢失	否	否



调整发送速率

- ✎ 如果两个流采用相同的方法（**加法或乘法**）调整发送速率，则用户都难以收敛到同时兼顾公平和效率的最佳发送速率



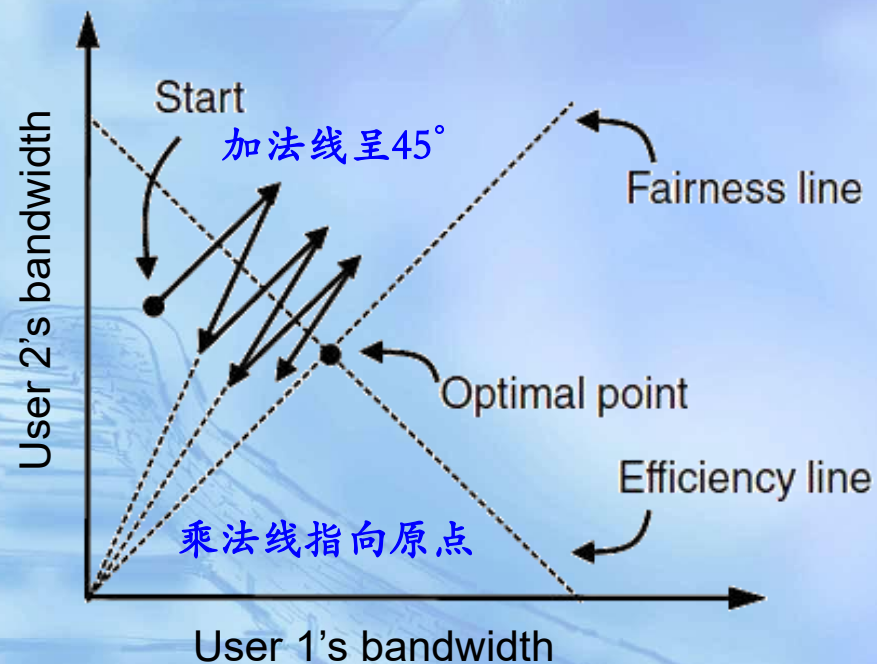


调整发送速率

✎ **AIMD** (加法递增乘法递减) : 没有收到拥塞信号时, 加法递增带宽; 收到拥塞信号时, 乘法递减带宽

✎ 可以收敛到兼顾公平和效率的最佳发送速率

✎ **AIMD**是TCP采用的拥塞控制算法



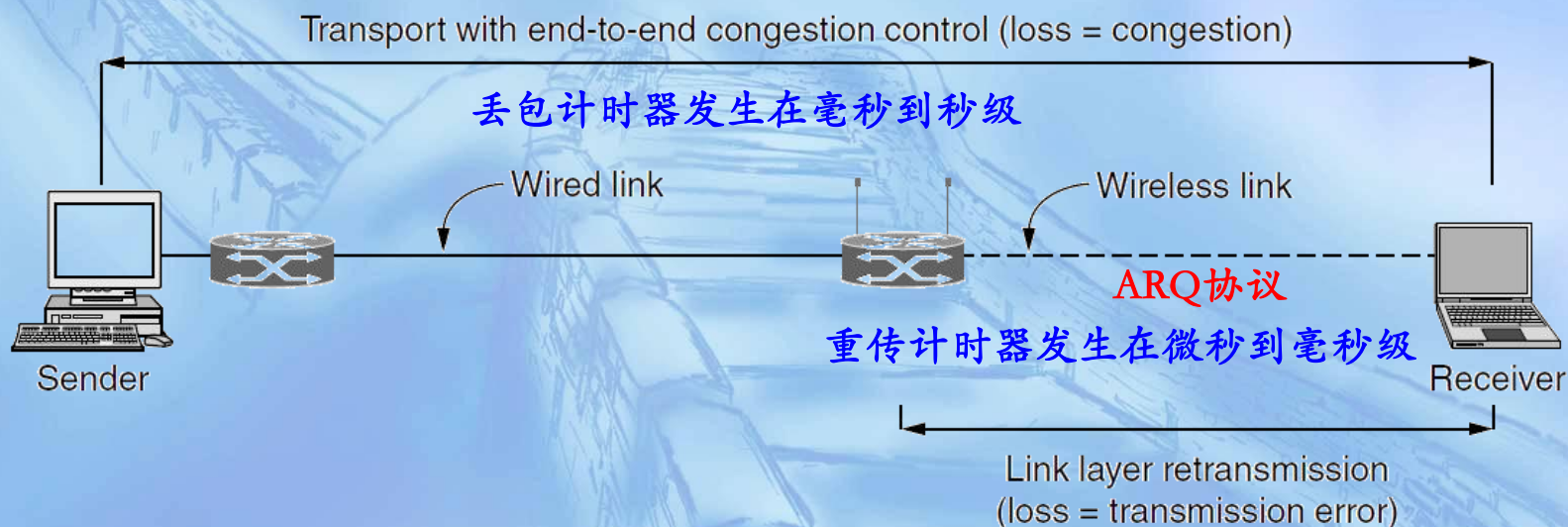


无线问题

诸如**TCP**的传输层协议是专为有线链路设计的，如果用在无线链路会有问题。

✎ **问题**：传输协议一般将丢包视作拥塞发生的信号，而在无线网络中丢包几乎都是因为传输错误引起的

✎ **解决**：**屏蔽策略**（无线链路使用**ARQ**掩盖错误）





用户数据报协议 (UDP)

UDP概述

应用1:RPC

应用2:RTP



概述

✍ UDP提供无连接、不可靠的数据报服务

- ✍ 数据传输无需建立和释放连接;
- ✍ 每个UDP用户数据报不需编号且独立传输;
- ✍ 无差控和流控功能。

✍ UDP只在IP的数据报服务之上增加了很少一点的功能: 端口的功能

✍ RFC 768描述了UDP



UDP用户数据报





格式解 释

首部

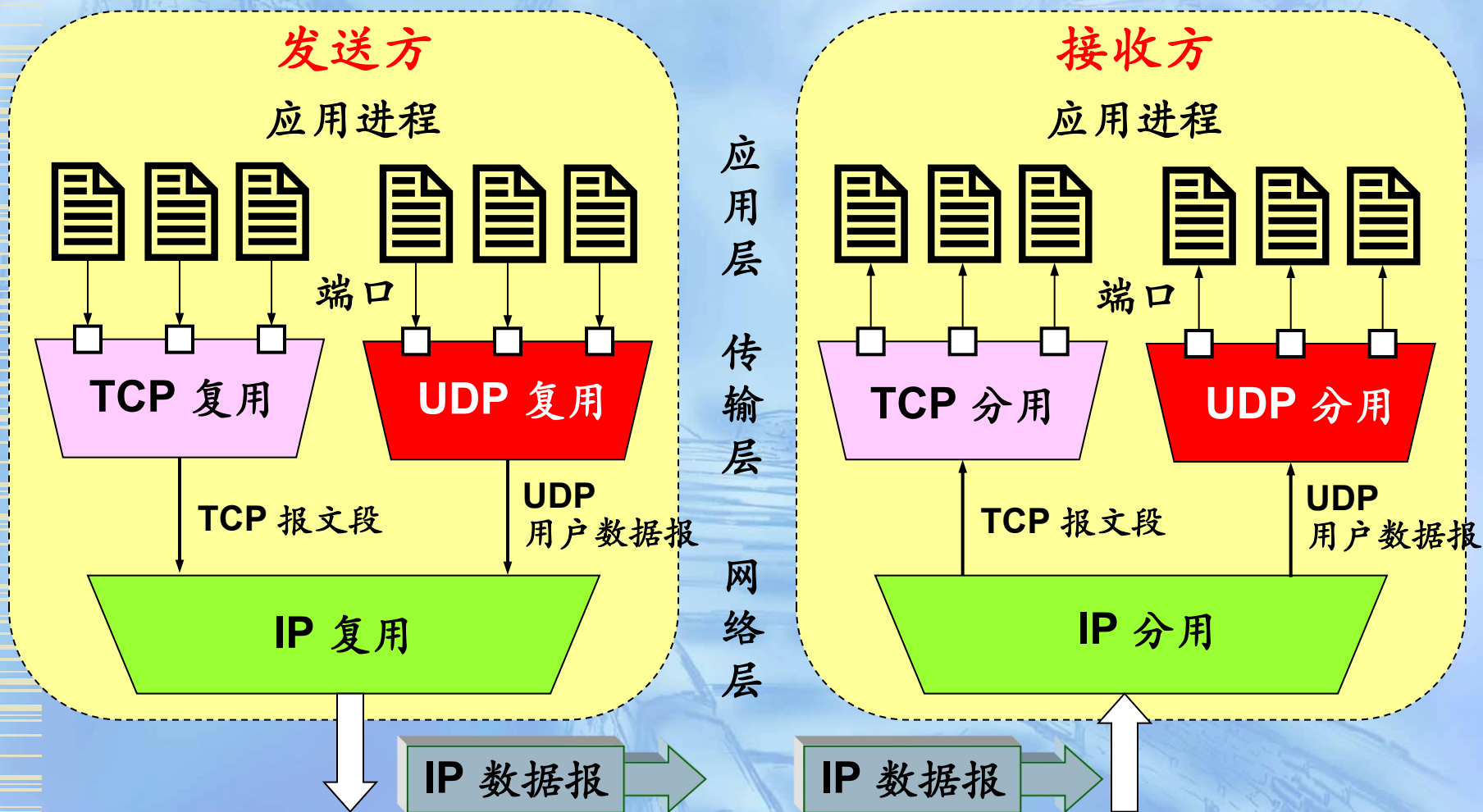
- ✦ **源端口**：2B，标明发送方进程地址；
- ✦ **目的端口**：2B，标明接收方进程地址；
- ✦ **长度**：2B，UDP数据报的总长度（首部+数据），8~65515B（由IP数据报限制）；
- ✦ **校验和**：2B（可选），防止UDP数据报在传输中出错。在计算校验和时，临时把“伪首部”和UDP用户数据报连接在一起。伪首部仅仅是为了计算校验和。

数据

- ✦ 高层交付的数据。



端口的作用





端口的使用

✍ 端口用一个 16 bit 端口号进行标志

端口号只具有本地意义，即端口号只是为了标志本计算机应用层中的各进程。在Internet中不同计算机的相同端口号是没有联系的。

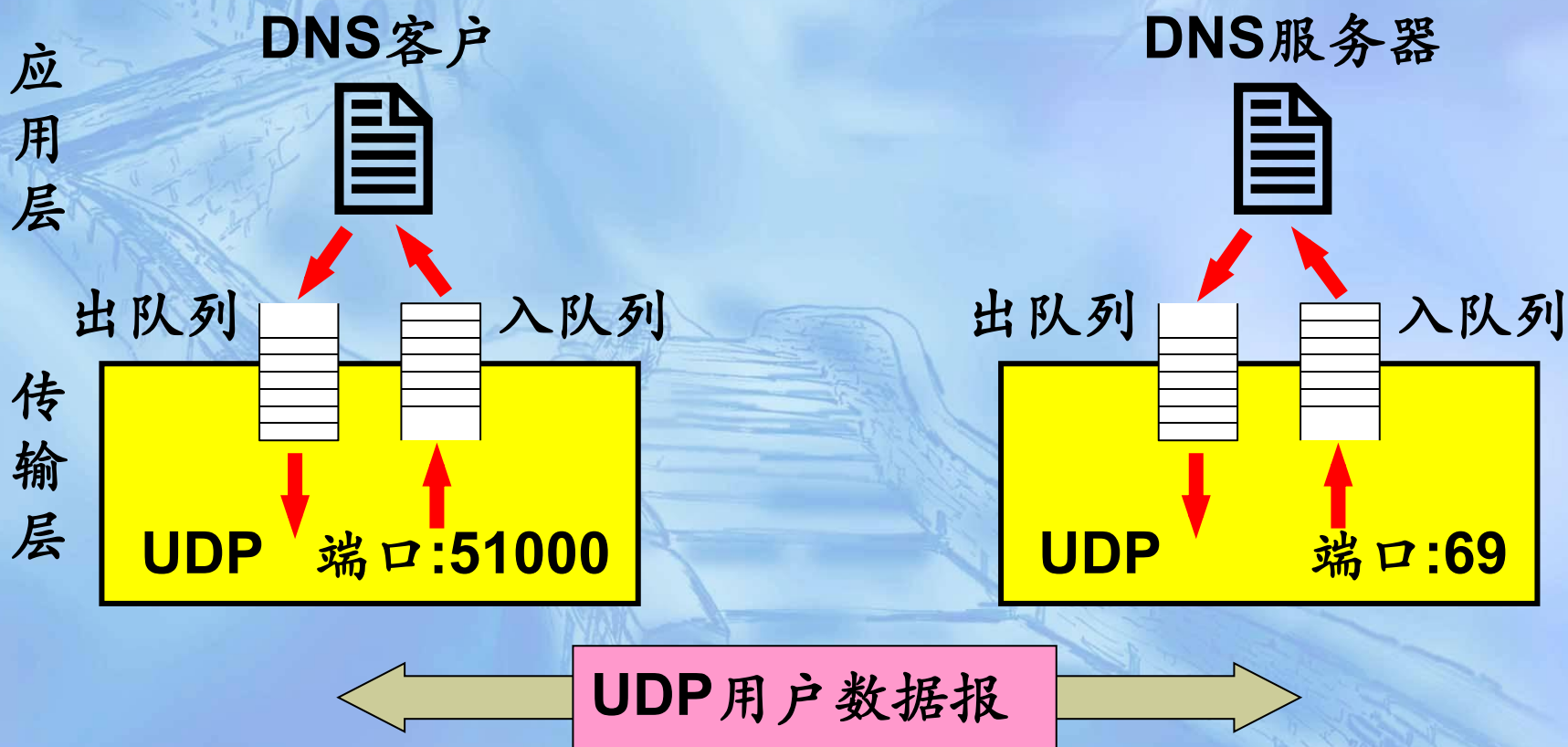
✍ 端口地址的分类

- ✦ 熟知端口：0~1023，一般分配给服务器进程；
- ✦ 登记端口：1024~49151，为没有熟知端口的应用程序使用的。使用这个范围的端口号必须在 IANA 登记，以防止重复；
- ✦ 客户端口或短暂端口：49152~65535，留给客户进程选择暂时使用。



端口的实现

UDP与应用层之间的端口都是用**报文队列**来实现。





总结

✍️ 特点

- 发送数据之前不需要建立连接;
- 无差错控制 (重传)、流量控制和拥塞控制;
- 通过端口复用多个进程;
- 可选的端-端差错检测功能。

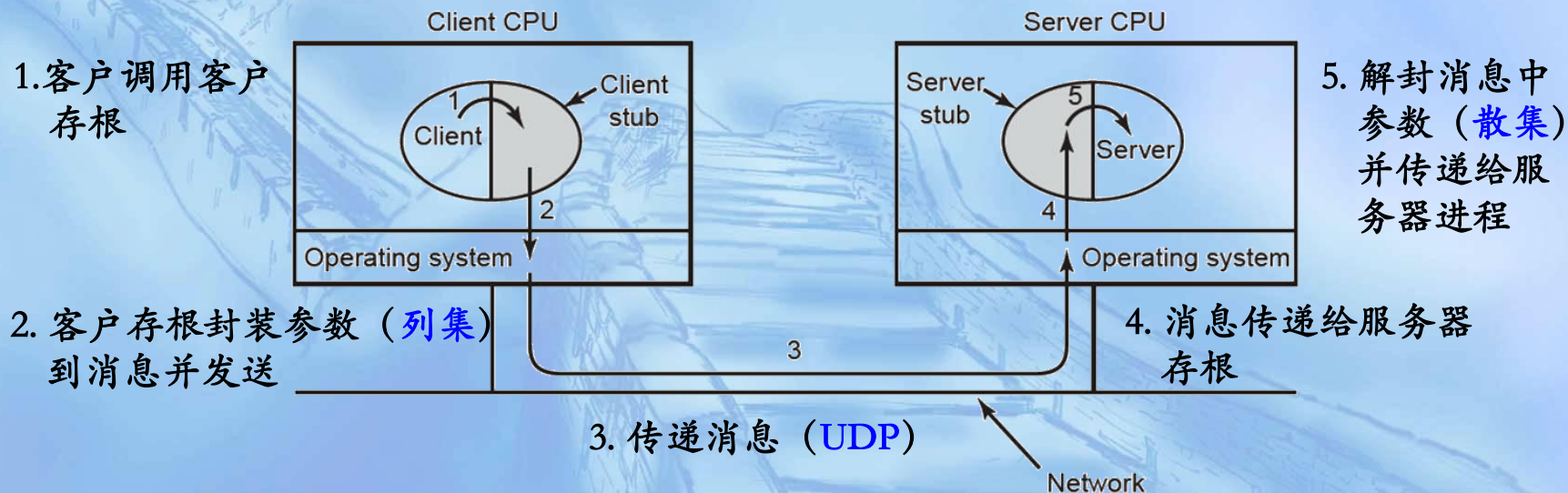
✍️ 应用

- ✦ 客户-服务器应用开发, 例如**RPC**;
- ✦ 适合具有内部流控和差控的进程, 例如**TFTP**;
- ✦ 适合于多播和广播;
- ✦ 适合于实时多媒体应用, 例如**RTP**;
- ✦ 可用于管理进程, 例如**SNMP**;



远程过程调用 (RPC)

- ✍ **远程过程调用 (RPC)**：本地程序调用远程主机上的过程，就像本地过程调用一样。调用过程称为**客户**，被调过程称为**服务器**。已经成为很多网络应用的基础。
- ✍ **UDP**可用于实现RPC，可提供一个**低延迟**的传输服务

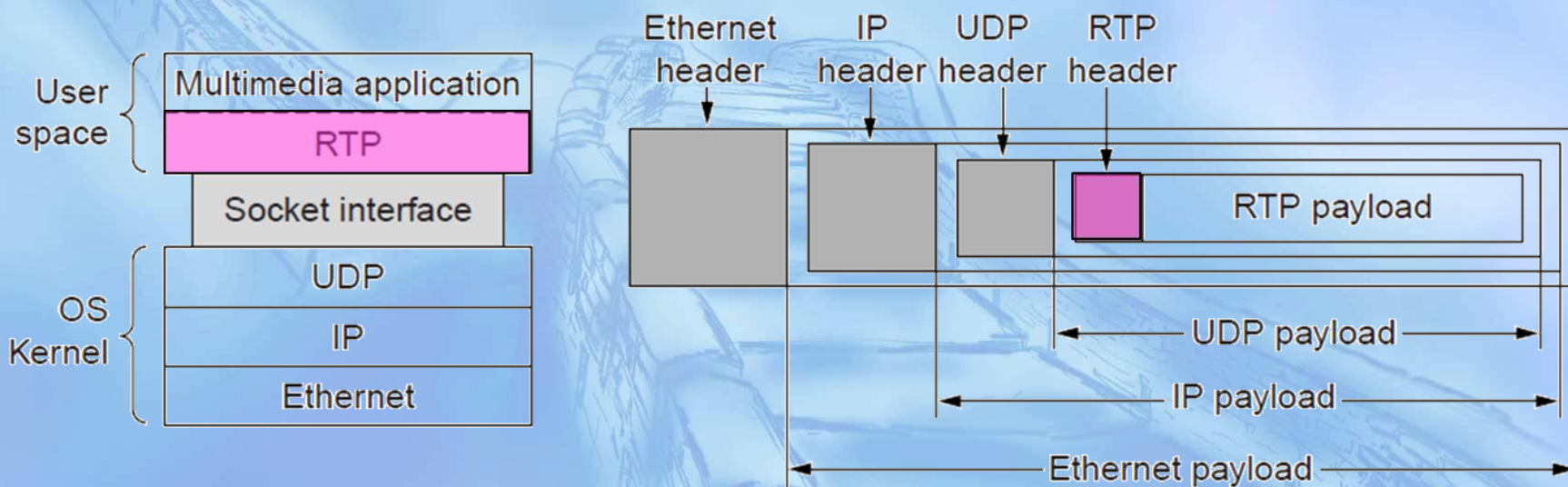


参数以消息形式由客户传递到服务器，执行结果的返回类似



实时传输协议 (RTP)

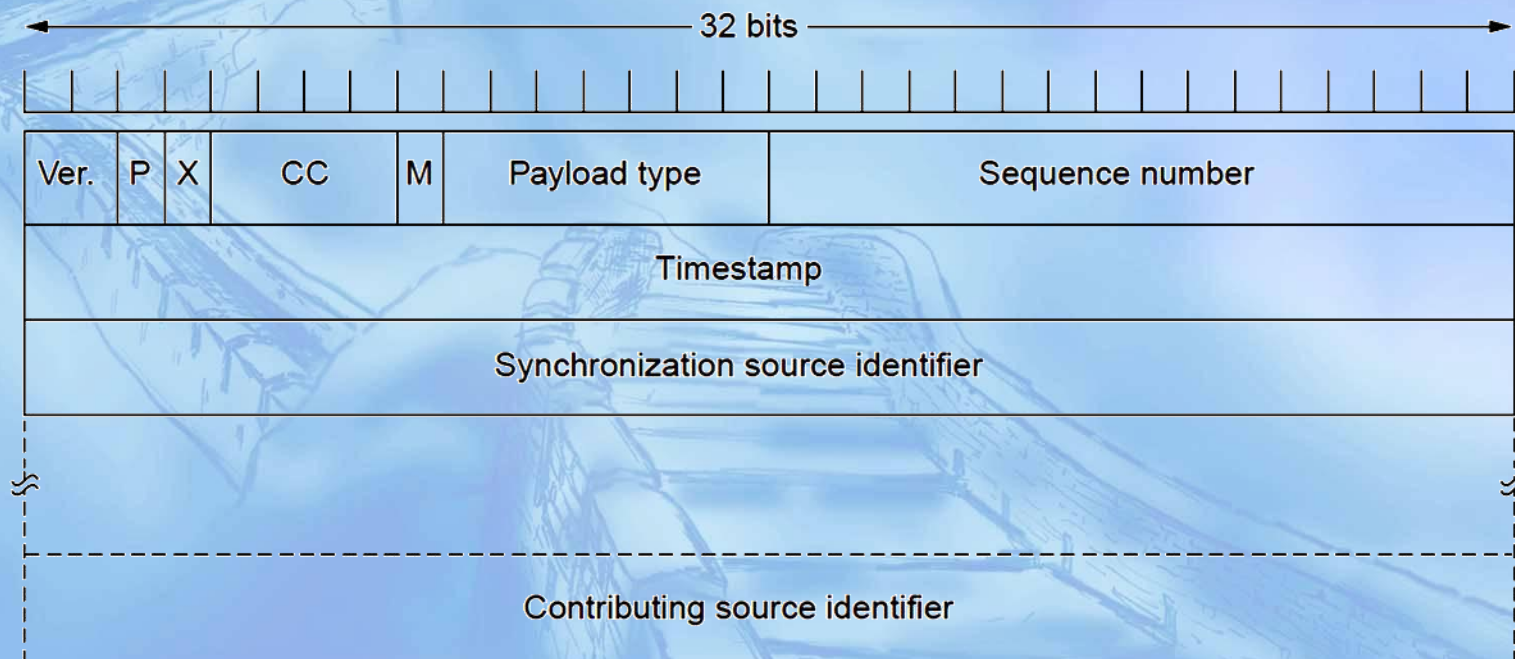
- ✍ **实时传输协议 (RTP)** 基于**UDP**为实时应用提供端-端的传输，但不提供任何服务质量的保证。
- ✍ 由**RFC 3550**描述，目前已广泛应用于多媒体应用：
 - ✦ **RTP协议**以数据报形式传输音频和视频数据
 - ✦ **RTCP协议**提供控制功能





RTP数据报

RTP协议的基本功能是将几个实时流复用到一个UDP数据报流中，没有确认，没有请求重传的机制。





格式解释

- ✍ **版本**: 2bit, 目前是版本2;
- ✍ **填充P**: 1bit, P=1表示有若干填充字节, 数据部分的最后一个字节用来表示填充的字节数;
- ✍ **扩展X**: 1bit, X=1表示有扩展首部 (很少使用);
- ✍ **参与源数CC**: 4bit, 用于指出有多少个源参与;
- ✍ **标记M**: 1bit, M=1表示本数据报具有特殊意义, 例如, 用来标记一个视频帧的开始等;
- ✍ **有效载荷类型**: 7bit, 用户指出RTP数据属于何种格式的应用, 例如, 音频有效载荷: μ 律PCM(0)、GSM(3)、A律PCM(8)等, 视频有效载荷: 活动JPEG(26)、MPEG1(32)、MPEG2(33)等;



格式解释

- ✍ **序号**: 16bit, 用于检测RTP数据报的丢失, 同时也能重新排列失序RTP数据报;
- ✍ **时间戳**: 32bit, 用于指出RTP数据报中数据的第一个字节的产生时间, 可用于消除时延的抖动, 也可用来使视频应用中声音和图像的同步;
- ✍ **同步源标识符**: 32bit, 用于指出RTP数据报属于哪一个流。利用本字段可以将多个数据流复用到一个UDP数据报流中, 或者从一个UDP数据报流中分用出多个数据流;



实时传输控制协议 (RTCP)

- ✍ **RTCP是与RTP配合使用的协议**
- ✍ **RTCP协议的主要功能：**
 - ✦ 服务质量的监视与反馈
 - ✦ 媒体间的同步
 - ✦ 多播组中成员的标识
- ✍ **RTCP数据报不对声音或图像数据进行封装**
- ✍ **RTCP数据报使用UDP传送，可将多个RTCP数据报封装在一个UDP用户数据报中**
- ✍ **RTCP数据报周期性地在网上传送，它带有发送端和接收端对服务质量的统计信息报告**



通过缓冲控制抖动

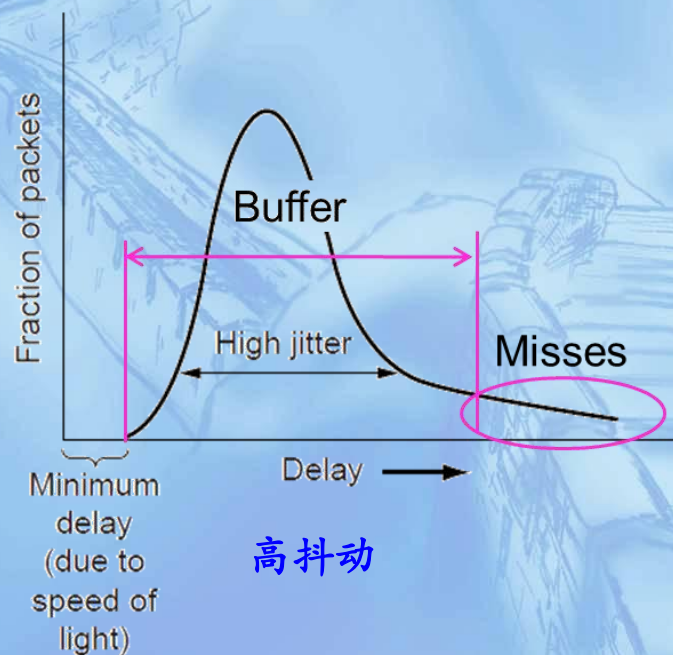
- ✎ **抖动**: 在发送端数据报以正确的时间间隔被依次注入网络, 但它们到达接收端的相对时间不同, 这种延迟的变化称为抖动
- ✎ **缓冲**: 在接收端播放媒体之前对其进行缓冲, 以此来减少抖动, 平滑播出



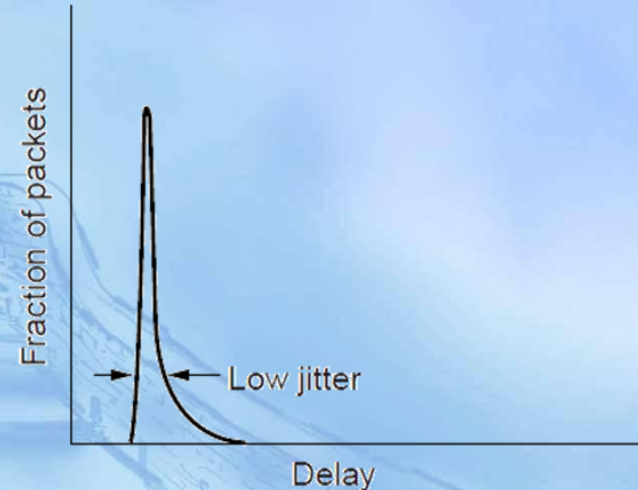


缓冲区大小

- ✗ 高抖动，或多变化的延迟，需要一个更大的缓冲区以避免错过播放；
- ✗ 传播延迟不影响缓冲区大小；



高抖动



低抖动



传输控制协议 (TCP)

✍ TCP概述

✍ TCP报文段

✍ 差错控制

✍ 流量控制

✍ 拥塞控制

✍ 连接管理



TCP 概述

TCP协议对上层提供**面向连接**、**端-端**的**可靠通信**服务。
服务特点有：

✎ 流式数据服务

提供字节流（字符流）数据服务，不保持报文的边界。
为进行流式服务，发送TCP和接收TCP都要使用**缓存**。

✎ 全双工服务

数据可在同一时间内双向流动。全双工服务为确认信息的**捎带**提供了基础。

✎ 可靠交付服务

数据传输可靠、按序、无丢失和无重复等。



TCP 的连接

- ✗ TCP把**连接**作为最基本的抽象;
- ✗ 每一条TCP连接有两个**端点**;
- ✗ 端点叫做**套接字(socket)**或**插口**, 由端口号拼接到IP地址构成:

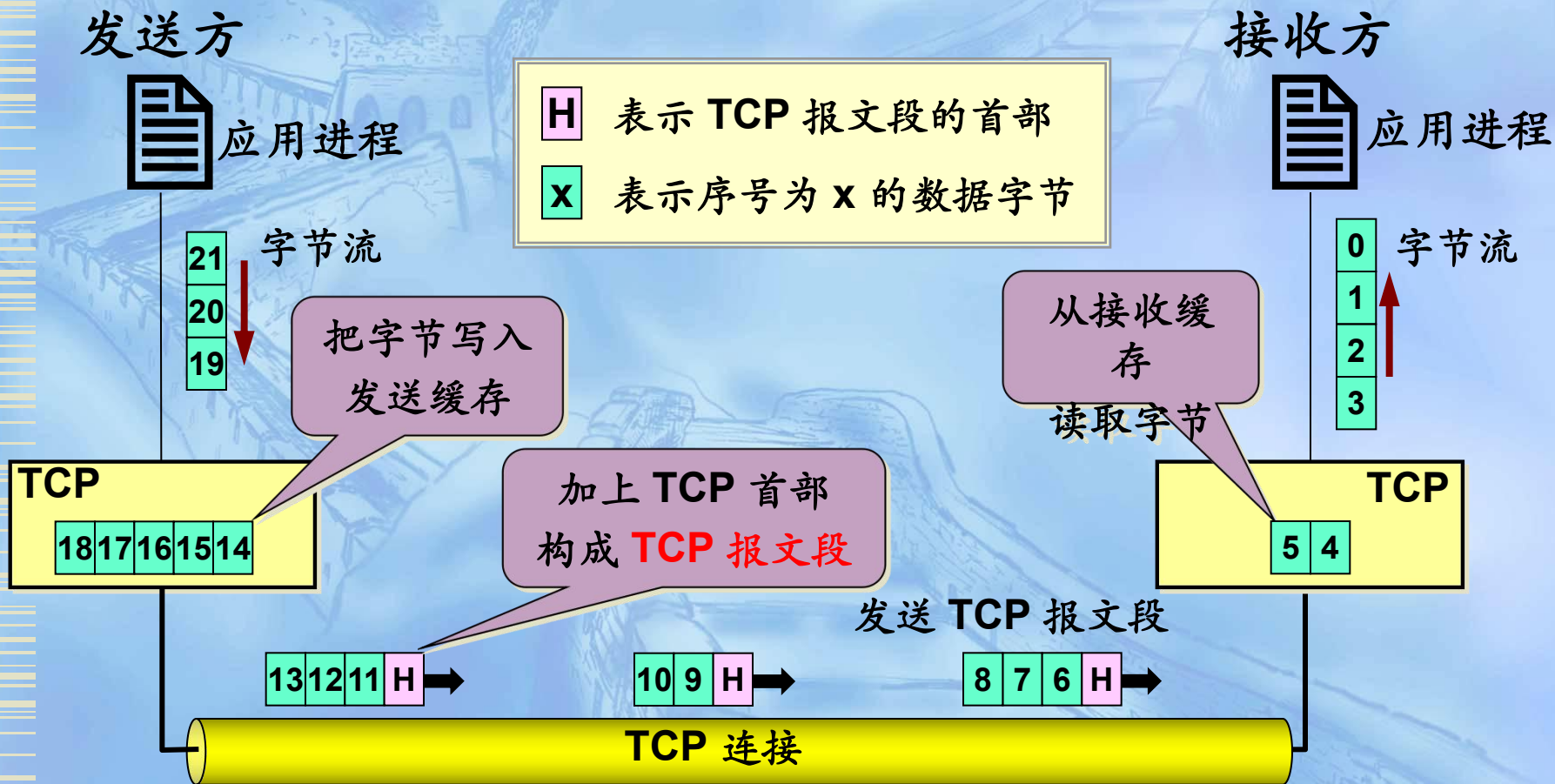
套接字 (socket) = (IP地址 : 端口号)

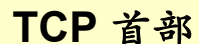
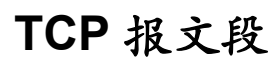
- ✗ 每一条 TCP 连接唯一地被通信两端的两个端点 (即两个套接字) 所确定。即:

TCP 连接 ::= {socket1, socket2}
= {(IP1: port1), (IP2: port2)}



TCP 的流式服务







格式解释 之一

✍ 端口

源端口和目的端口（各2B）。端口是传输层与应用层的服务接口。传输层的复用和分用功能都要通过端口才能实现。

✍ 序号

4B。TCP连接中传送的数据流中的每一个字节都编上一个序号。序号字段的值则指的是本报文段所发送的数据的第一个字节的序号。

✍ 确认号

4B，是期望收到对方的下一个报文段的数据的第一个字节的序号。



著名的熟知端口

Port	Protocol	Use
20, 21	FTP	File transfer
22	SSH	Remote login, replacement for Telnet
25	SMTP	Email
80	HTTP	World Wide Web
110	POP-3	Remote email access
143	IMAP	Remote email access
443	HTTPS	Secure Web (HTTP over SSL/TLS)
543	RTSP	Media player control
631	IPP	Printer sharing



格式解释 之二

✍ 首部长度的

4bit, TCP报文段的首部长度的 (4B为计算单位), 长度范围[20B, 60B]。

✍ 保留

4bit, 保留为今后使用, 但目前应置为 0。

✍ 显式拥塞通知 (ECN)

当TCP接收端收到来自网络的拥塞提示后, 就设置**ECE**以便给TCP发送端发ECN-Echo信号, 告诉发送端放慢发送速度; TCP发送端设置**CWR**以便给TCP接收端发**CWR**信号, 这样接收端就知道发送端已经放慢速率, 不必再给发送端发ECN-Echo信号。



格式解释 之三

✎ 紧急比特URG

URG =1 表明紧急指针字段有效。它告诉系统此报文段中有紧急数据（放在**最前面**），应尽快传送（相当于高优先级的数据），而不要按原来的排队顺序来传送。本字段需与“**紧急指针**”字段（**16bit**。用于指出紧急数据的字节数，即指出紧急数据的**末尾**）配合使用。

✎ 确认比特ACK

ACK =1时“确认号”字段有效，**ACK=0**时“确认号”字段无效。

✎ 推送比特PSH

接收TCP收到**PSH=1**的报文段，就尽快地交付给接收应用进程，而不再等到整个缓存都填满了后再向上交付。



格式解释 之四

✎ 复位比特 **RST**

RST=1表明TCP连接中出现严重差错（如由于主机崩溃）必须释放连接，然后再重新建立传输连接。

✎ 同步比特 **SYN**

在连接建立时用来同步序号。当**SYN=1**且**ACK=0**表示这是一个连接请求报文段，当**SYN=1**且**ACK=1**表示这是一个连接接受报文段。

✎ 终止比特 **FIN**

用来释放一个连接。**FIN=1**表明此报文段的发送端的数据已发送完毕，并要求释放传输连接。



格式解释 之五

✍ 窗口字段

2B。用来控制对方发送的数据量（单位为1B）。TCP连接的一端根据设置的缓存空间大小确定自己的接收窗口大小，然后通知对方以确定对方的发送窗口的上限。

✍ 检验和

2B。检验范围包括首部和数据两部分。在计算检验和时，要在TCP报文段的前面加上12B的伪首部（同UDP，但应将伪首部的第4个字段由17改为6，第5字段中的UDP长度改为TCP长度）。



格式解释 之六

✎ **选项：** 长度可变

- ✎ **最大报文段长度 MSS 选项：** MSS 告诉对方 TCP：“我的缓存所能接收的报文段的数据字段的最大长度是 **MSS** 个字节。”；
- ✎ **窗口扩大选项：** 3B，其中有1B表示移位值 **S**（最大值为14）。新的窗口值等于TCP首部中的窗口位数增大到 $(16 + S)$ ，相当于把窗口值向左移动 **S** 位后获得实际的窗口大小；
- ✎ **时间戳选项：** 10B，其中最主要的字段时间戳值字段（4 B，记录报文段发送时间）和时间戳回送回答字段（4 B，复制报文段发送时间）；作用：计算往返时间RTT、防止序号绕回（PAWS）；



格式解释 之七

✍ **选项：**续前

.....

✍ **选择确认选项 (SACK)：**使得接收端可以告诉发送端已经接收报文段的序号范围，这是对确认号字段的补充。

✍ **填充**

这是为了使整个首部长度是 **4B** 的整数倍。

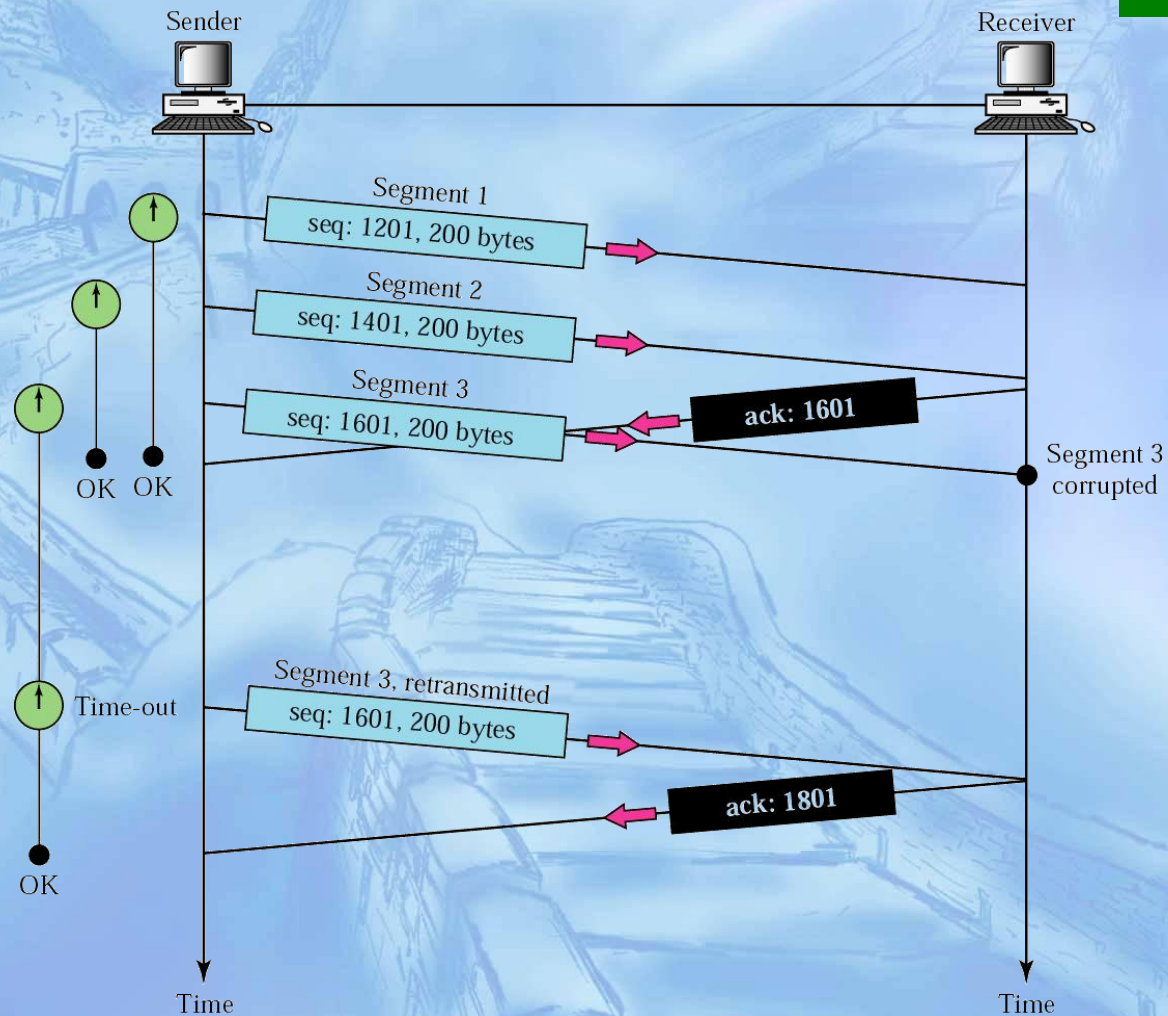


差错控制

TCP使用差错控制提供可靠性。差错控制的方法为：用报文段中的**校验和**字段进行检测差错是否发生，如果发生，丢弃该报文段，让发送端**超时重发**；如果没有发生，向发送端发送**确认**进行证实。

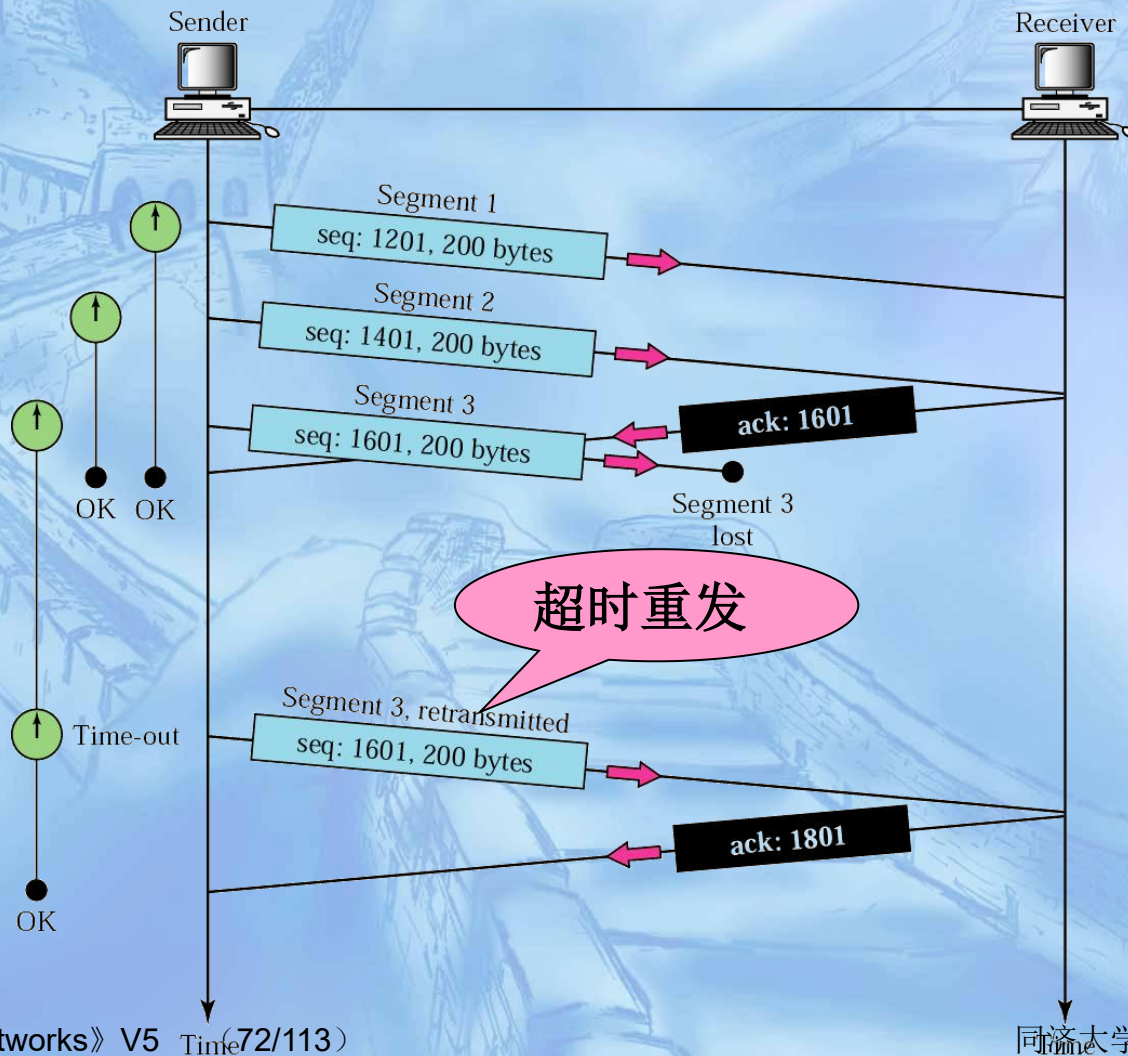


例:受损伤的报文段



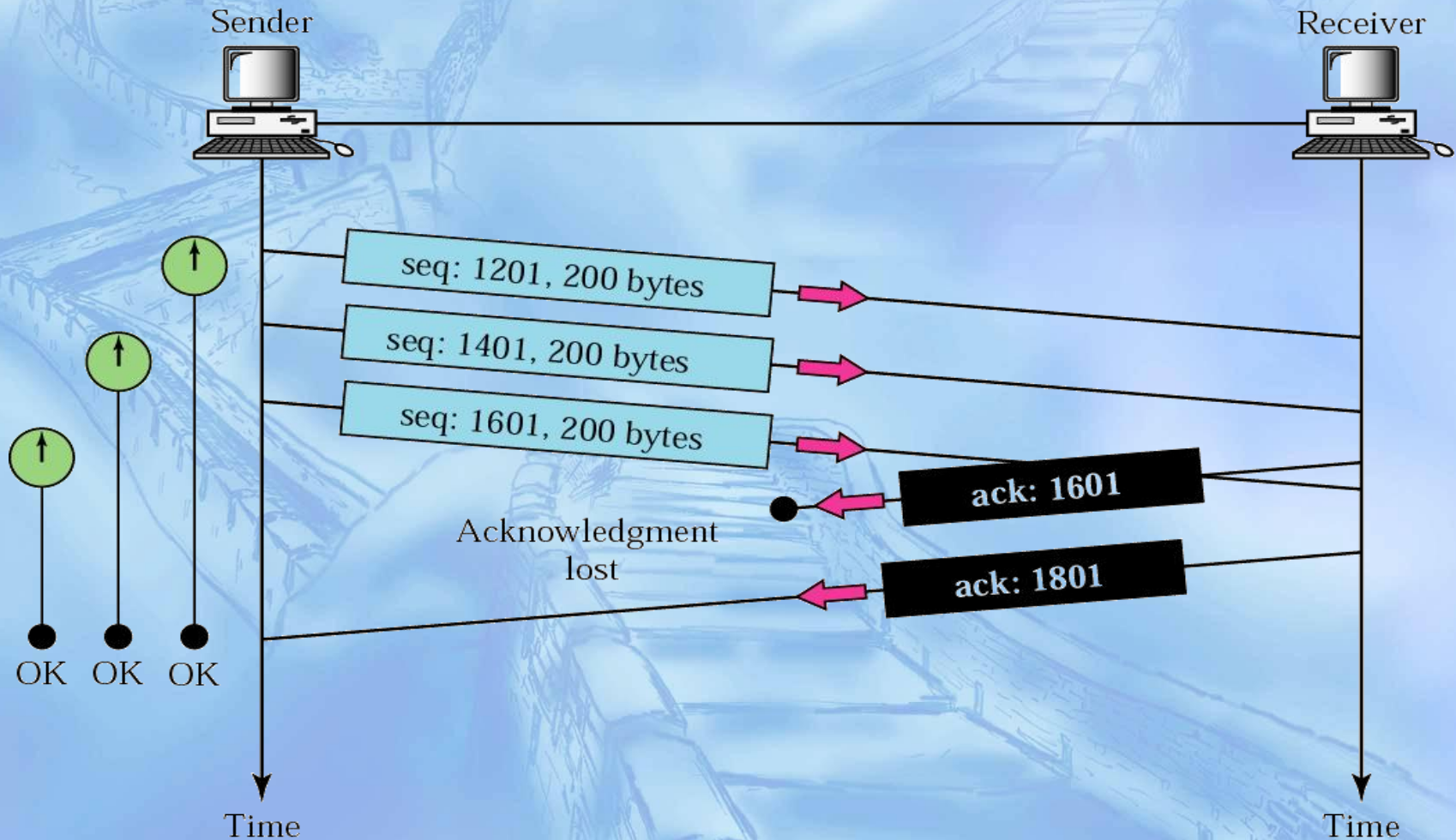


例:丢失的报文段





例:丢失的确认





例:其他情况

重复的报文段

处理方法：目的TCP丢弃重复的报文段

。

失序的报文段

处理方法：目的TCP对失序的报文段不确认，直到收到所有它以前的报文段为止。



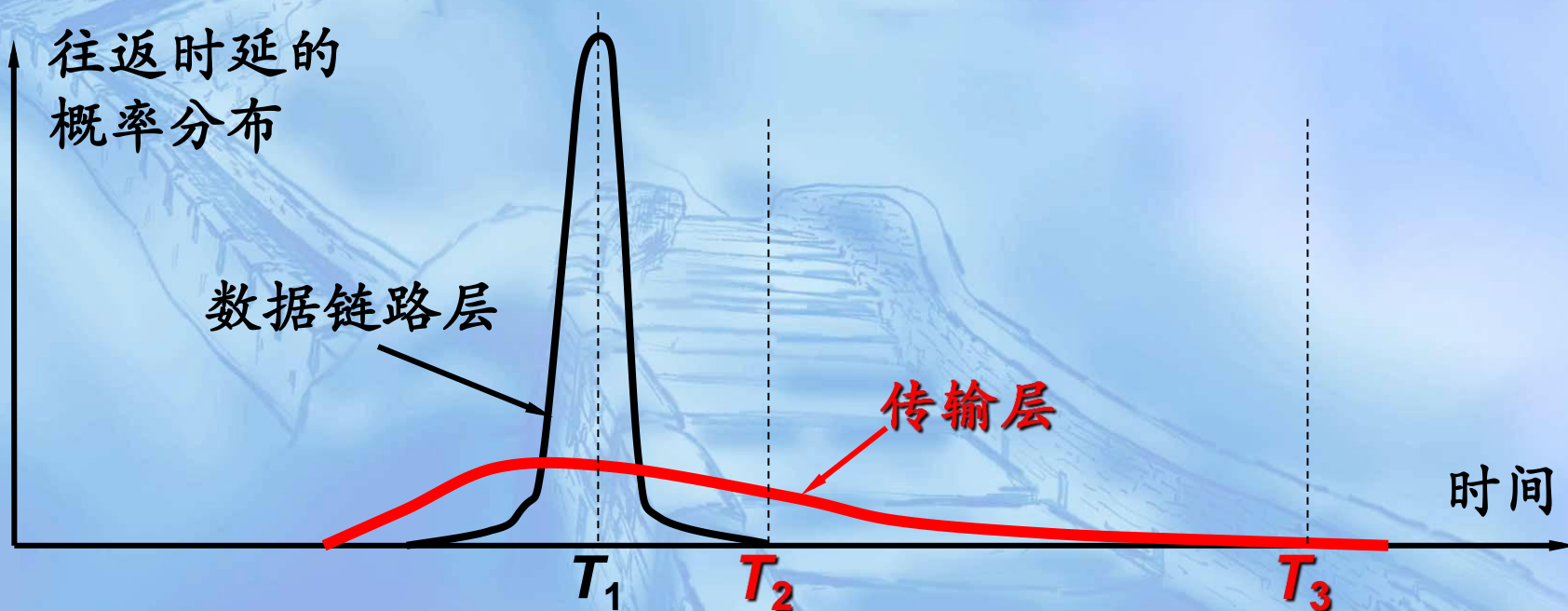
重传计时器

TCP 每发送一个报文段，就对这个报文段设置一次计时器。如果在计时器截止时间到之前收到对此特定报文段的确认，则撤消此计时器，否则就要重传此报文段，并将计时器复位。



如何确定重传时间？

由于TCP的下层是一个互连网环境，IP数据报所选择的路由变化很大。因而传输层的往返时延（RTT）的方差也很大。





往返时延的自适应算法

- ✎ TCP保留了 RTT 的一个**加权平均往返时间 SRTT**（这又称为**平滑的往返时间**）；
- ✎ 第一次测量到 RTT 样本时，SRTT 值就取为所测量到的 RTT 样本值。以后每测量到一个新的 RTT 样本，就按下式重新计算一次 SRTT：

$$\text{SRTT} = \alpha \times \text{SRTT} + (1 - \alpha) \times \text{新的 RTT 样本}$$

式中， $0 \leq \alpha < 1$ 。若 α 很接近于零，表示 RTT 值更新较快。若选择 α 接近于 1，则表示 RTT 值更新较慢。

- ✎ **RFC 6298** 推荐的 α 值为 **7/8**。



超时重传时间 (RetransmissionTime-Out)

- ✗ RTO应略大于上面得出的加权平均往返时间 SRTT;
- ✗ RFC 6298 建议使用下式计算 RTO:

$$\text{RTO} = \text{SRTT} + 4 \times \text{RTTVAR}$$

RTTVAR是RTT的偏差的加权平均值。

- ✗ RFC 6298 建议这样计算 RTTVAR。第一次测量时, RTTVAR值取为测量到的 RTT 样本值的一半。在以后的测量中, 则使用下式计算加权平均的 RTTVAR:

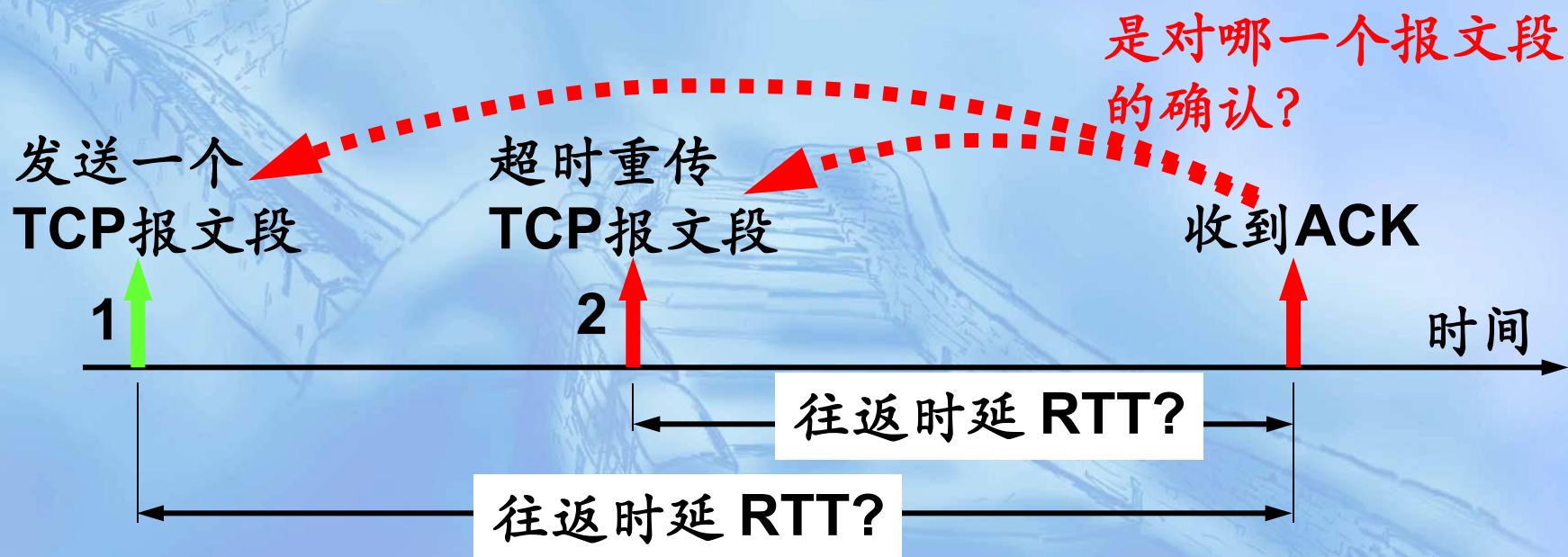
$$\text{RTTVAR} = \beta \times \text{RTTVAR} + (1 - \beta) \times |\text{SRTT} - \text{新的RTT样本}|$$

β 是个小于 1 的系数, 其推荐值是 3/4。



计算RTT中的问题

TCP 报文段 1 没有收到确认。重传（即报文段 2）后，收到了确认报文段 **ACK**。如何判定此确认报文段是对原来的报文段 1 的确认，还是对重传的报文段 2 的确认？





解决:Karn算法

✍ 算法

在计算平均往返时延RTT时，只要报文段重传了，就不采用其往返时延样本。这样得出的平均往返时延 RTT和重传时间就较准确。

✍ 问题

设想出现这样的情况：报文段的时延突然增大了很多。因此在原来得出的重传时间内，不会收到确认报文段，于是重传报文段。但根据Karn算法，不考虑重传报文段的RTT样本。这样，重传时间就无法更新。



解决:修正的Karn算法

报文段每重传一次，就将重传时间增大一些：

新的重传时间 = $\gamma \times$ (旧的重传时间)

系数 γ 的典型值是2。当不再发生报文段的重传时，才根据报文段的往返时延更新平均往返时延 **RTT** 和重传时间的数值。

实践证明，这种策略较为合理。



流量控制

✍ 问题

发送方发送能力大于接收方接收能力，会导致接收方来不及接收。

✍ 解决

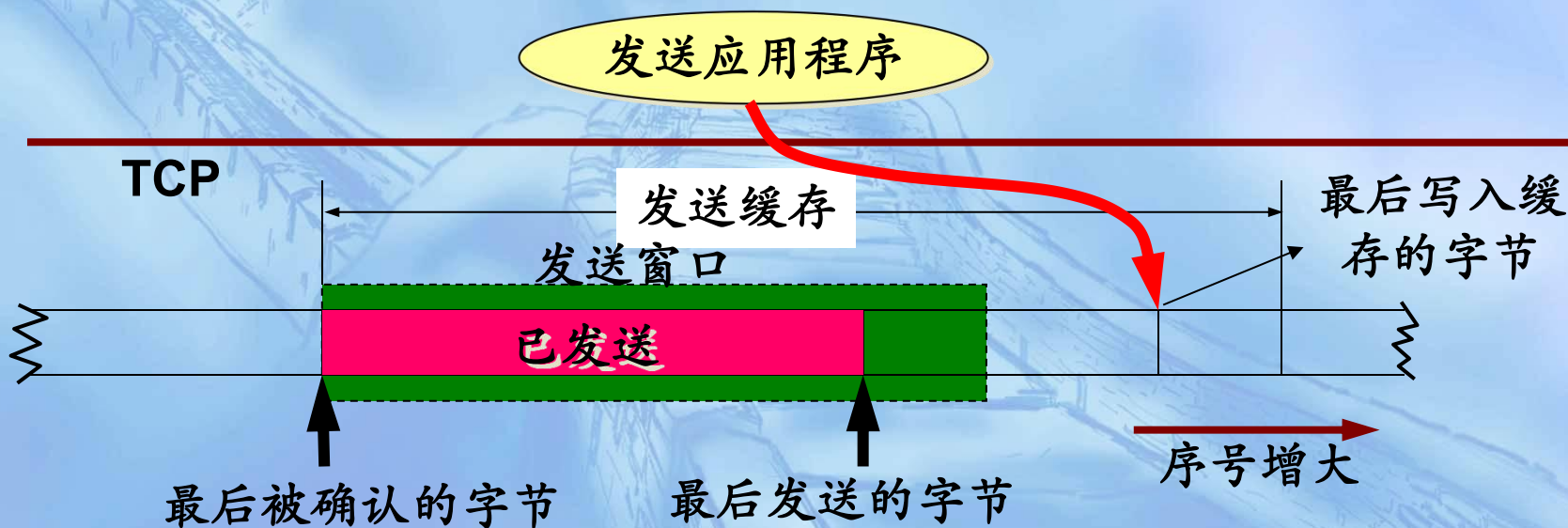
控制发送方的速率，使其不超过接收方所能承受的能力，常采用滑动窗口机制进行控制。



滑动窗口

✎ 发送窗口

每个发送端设置一个，用于表示发送端可以发送数据的大小。发送窗口的大小在连接建立时由双方商定，在数据传输过程中取决于接收端。

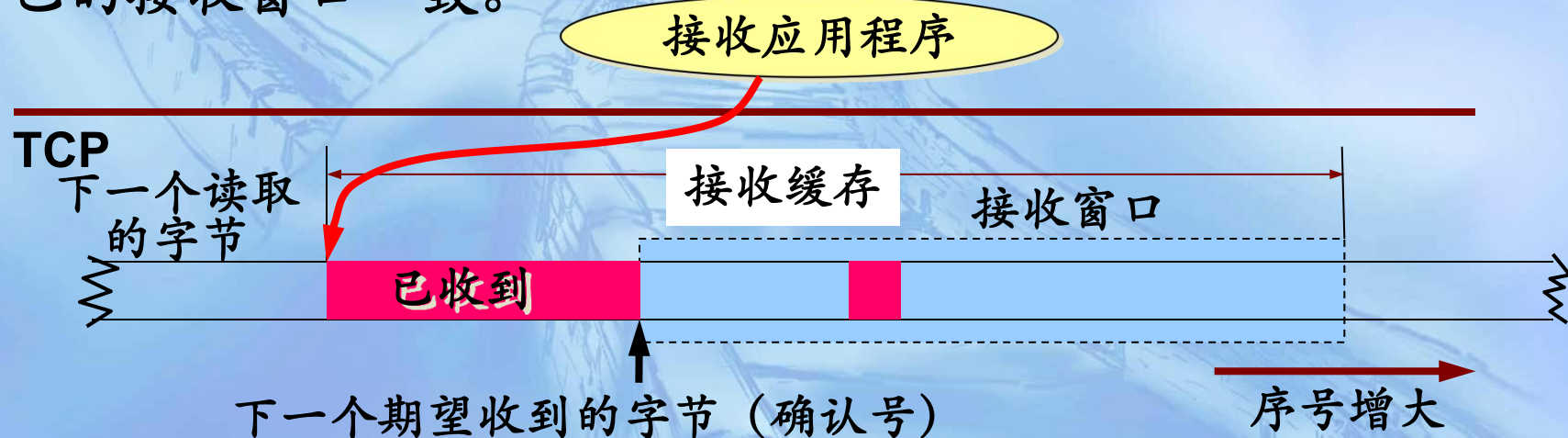




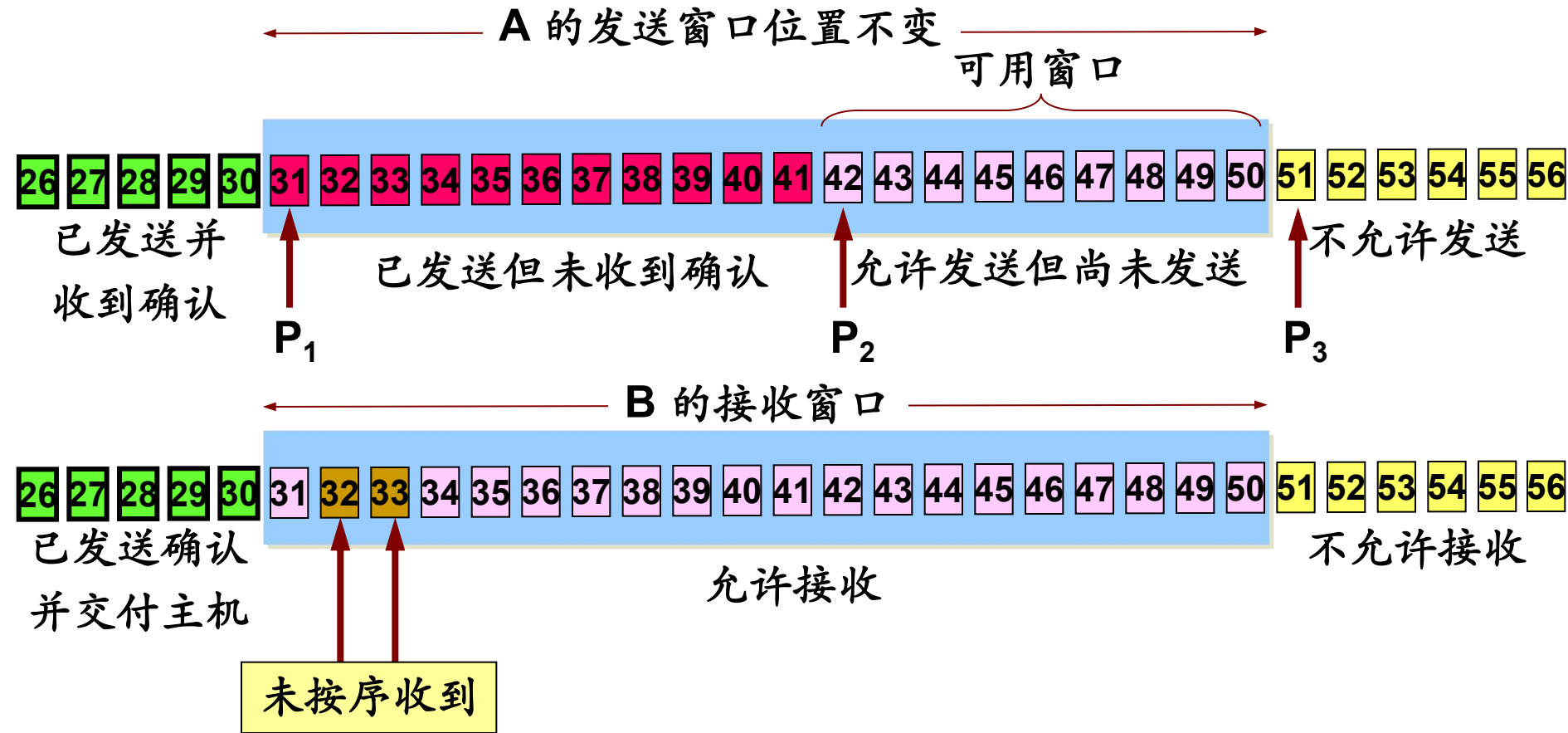
滑动窗口

接收窗口

每个接收端设置一个，用于表示接收端允许接收的数据的大小。接收窗口的大小在连接建立时由双方商定，在数据传输过程中，接收端可根据自己的资源情况，动态调整接收窗口的大小，然后告诉对方，使对方的发送窗口与自己的接收窗口一致。



A 发送了 11 个字节的数据



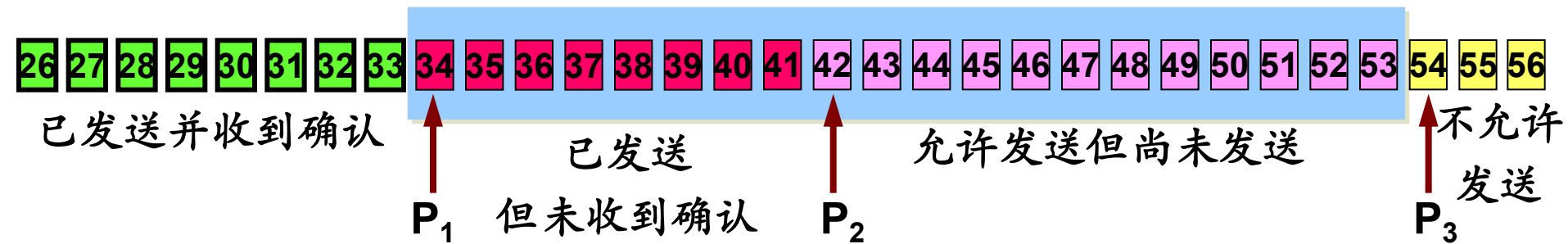
$P_3 - P_1 = A$ 的发送窗口 (又称为**通知窗口**)

$P_2 - P_1 =$ 已发送但尚未收到确认的字节数

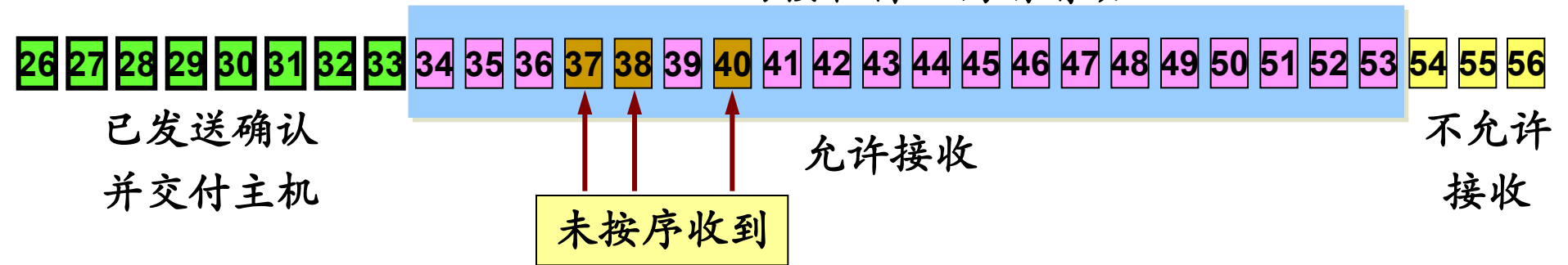
$P_3 - P_2 =$ 允许发送但尚未发送的字节数 (又称为**可用窗口**)

A 收到新的确认号，发送窗口向前滑动

A 的发送窗口向前滑动 →



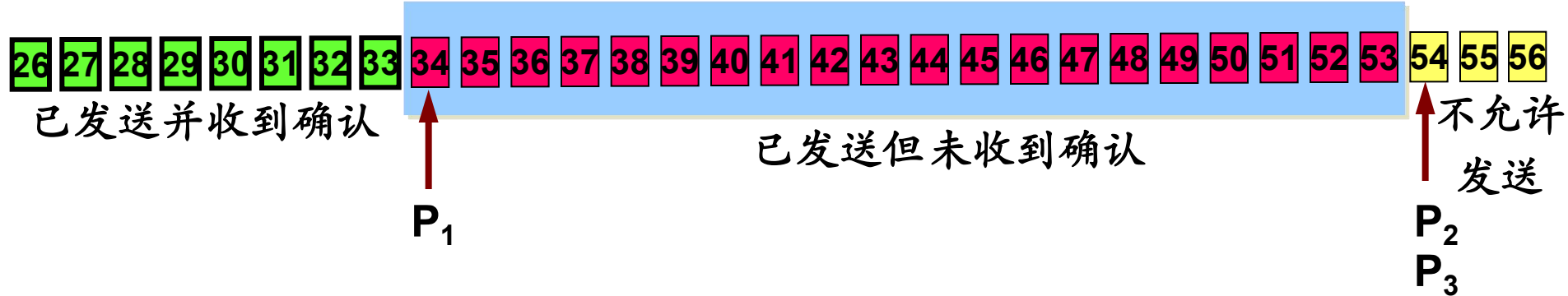
B 的接收窗口向前滑动 →



先存下，等待缺少的
数据的到达

**A 的发送窗口内的序号都已用完，
但还没有再收到确认，必须停止发送。**

A 的发送窗口已满，有效窗口为零



例:利用可变窗口大小进行流量控制

(TCP报文段长100B, 双方确定的最初窗口值是 400B)





滑动窗口的本质

上述滑动窗口的本质是通过**两个缓存**和**一个窗口**来实现的。发送端设置一个缓存，用于存储从应用程序来的数据，对这个缓存设置一个窗口，只要此窗口大小不是0就可以发送数据。接收端也设置一个缓存，用于存储刚接收到的数据，以便应用程序取走，接收端会将自身缓存剩余情况告知发送端，以便发送端改变窗口大小。



糊涂窗口综合症

当发送应用程序产生数据很慢，或接收应用程序吸收数据很慢，会使发送数据的报文段很小，引起操作效率的降低。



发送端

✎ 问题现象

例如：例如一次产生1B，会产生只包括1B数据的TCP报文段（40B的开销：20B的TCP首部和20B的IP首部），结果有很多41B的TCP报文段在Internet中传输。

✎ 解决方法

采用**Nagle算法**：① 发送端将从应用程序收到的第一块数据发送出去，哪怕只有一个字节。② 将后面到达的数据都缓存起来，当收到对方的确认或数据已积累到可以装成一个最大的报文段时，将缓存中的所有数据组装成一个报文段进行发送。③ 对剩余的传输，重复上一步骤。



接收端

✍ 问题现象

例如：一次吸收1B，也会使发送端产生只包括1B的TCP报文段（40B的开销：20B的TCP首部和20B的IP首部），结果有很多41B的TCP报文段在Internet中传输。

✍ 解决方法

可采用两个方法解决：① **Clark解决方法**：只要有数据到达就发送确认，但宣布的窗口大小为0，直至缓存空间已能放入最大长度的报文段，或缓存空间的一半已经空了。② **延迟的确认**：当一个报文段到达时并不立即发送确认，直到缓存有足够的空间为止。



拥塞控制

✍ 问题

部分或整个子网有太多的数据而造成性能下降的状况。

✍ 原因

内存配置不当、处理器太慢、带宽太低、缺乏综合治理等，本质原因是： $\sum \text{对资源的需求} > \text{可用资源}$ 。不能头痛医头，脚痛医脚，只有等网络中所有组件都相互平衡后，才能消除拥塞。



拥塞控制思想

✎ **思想：**控制发送方发送速率

发送端的窗口大小不仅取决于接收端（接收端通知的窗口大小），而且取决于网络的拥塞状况（拥塞窗口的大小），即：

发送窗口 = Min[通知窗口, 拥塞窗口]

✎ **通知窗口 (rwnd)：**是接收端根据其接收能力许诺的窗口值，是来自接收端的流量控制。接收端将通知窗口的值放在TCP报文段的首部中，传送给发送端。

✎ **拥塞窗口 (cwnd)：**是发送端根据网络拥塞情况得出的窗口值，是来自发送端的拥塞控制。



拥塞窗口的确定

(AIMD:加法增大乘法减小)

 慢开始和拥塞避免

 快重传和快恢复



慢开始和拥塞避免

✎ 增加策略

当一个连接初始化时，拥塞窗口设为1（一个报文段）。在拥塞发生前，拥塞窗口从1开始，在每收到一个对新的报文段的确认后，将拥塞窗口增加1（慢开始），即拥塞窗口以指数规律进行增长（1、2、4、8...）。当拥塞窗口大小到达门限（**ssthresh**）时，以线性规律进行增长（避免拥塞，加法增大）。

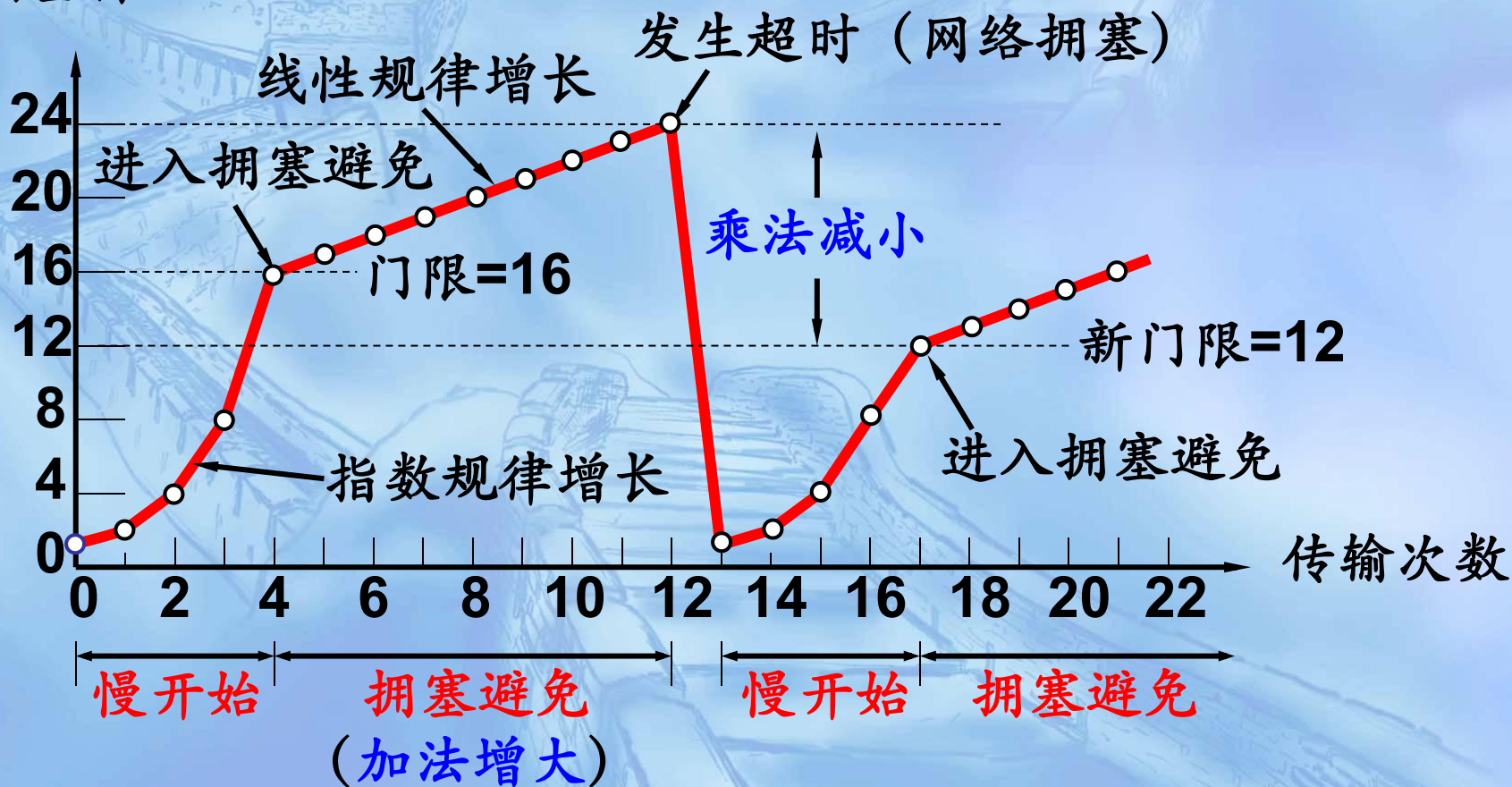
✎ 减少策略

如果出现拥塞（判断的依据是超时），窗口门限值设置为上次拥塞窗口大小的一半（乘法减小），而拥塞窗口降低到1，又从1开始增加。



图示

拥塞窗口cwnd



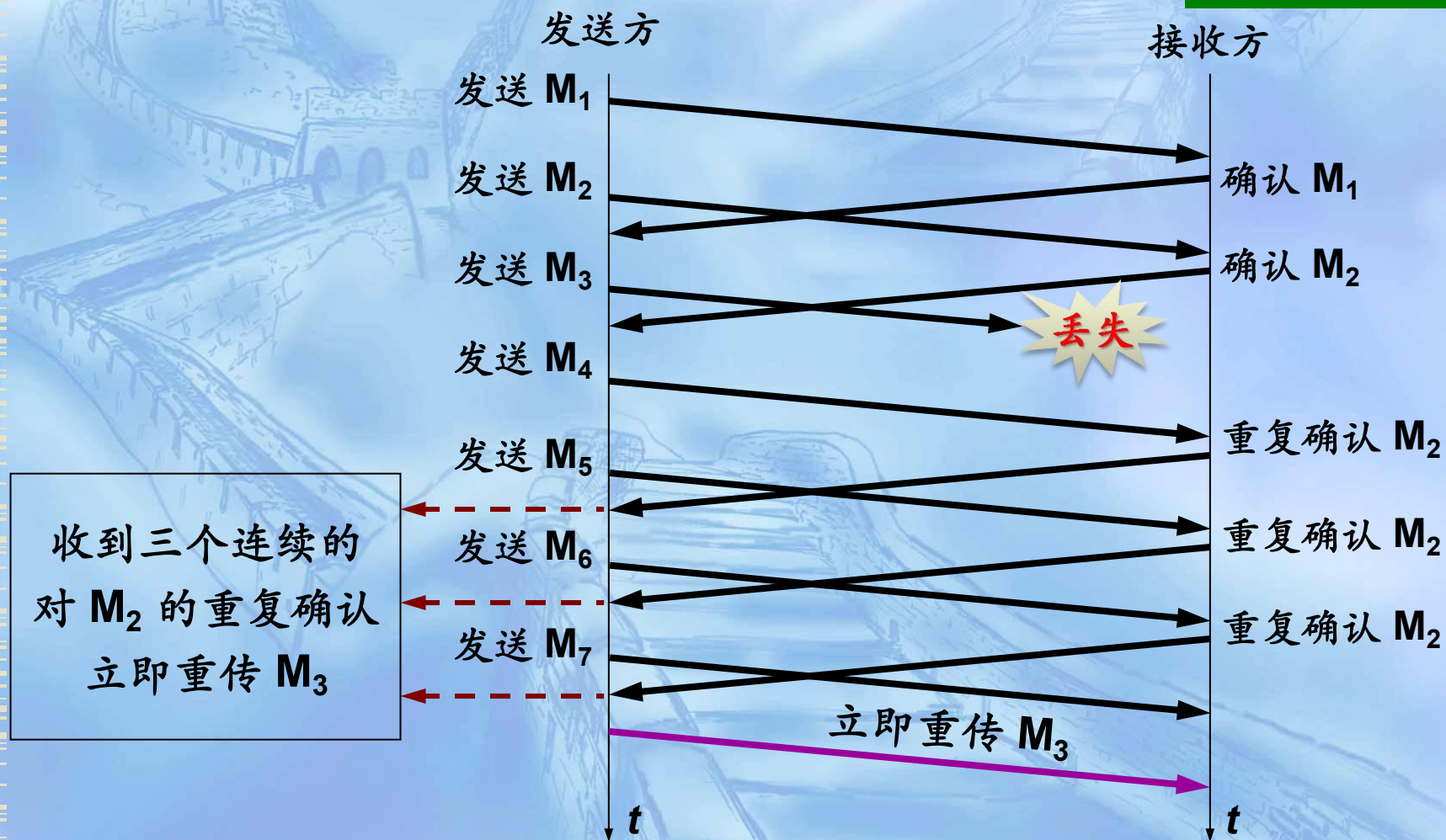


快重传

- ✎ 快重传算法首先要求接收方每收到一个失序的报文段后就立即发出重复确认。这样做可以让发送方及早知道有报文段没有到达接收方；
- ✎ 发送方只要一连续收到三个重复确认就应当立即重传对方尚未收到的报文段；
- ✎ 不难看出，快重传并非取消重传计时器，而是在某些情况下可更早地重传丢失的报文段，提高网络的吞吐量。



例:快重传



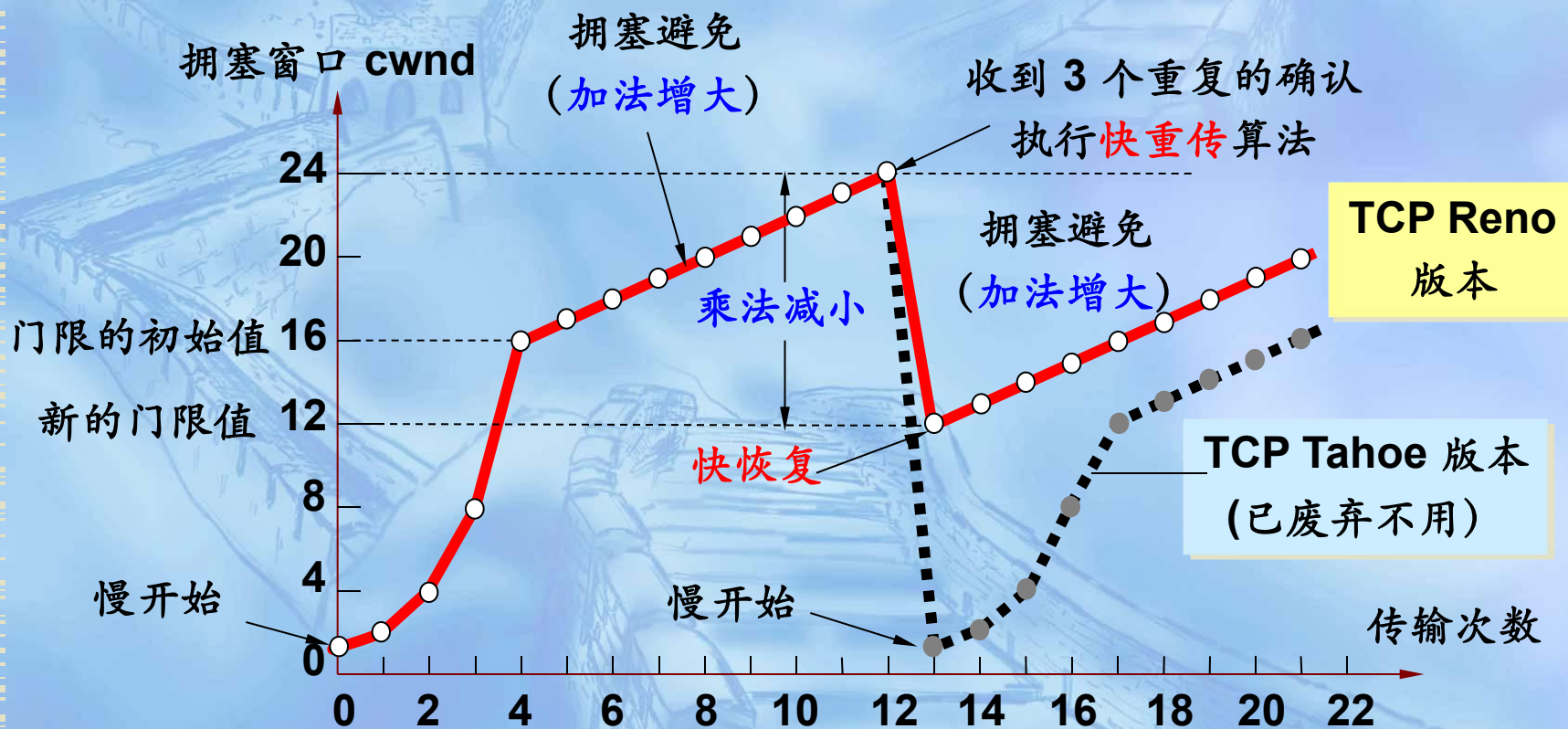


快恢复

- ✎ 当发送端收到连续三个重复的确认时，就执行“乘法减小”算法，把慢开始门限（**ssthresh**）减半。但接下去不执行慢开始算法；
- ✎ 由于发送方现在认为网络很可能没有发生拥塞，因此现在不执行慢开始算法，即拥塞窗口 **cwnd** 现在不设置为 1，而是设置为慢开始门限减半后的数值，然后开始执行拥塞避免算法（加法增大），使拥塞窗口缓慢地线性增大（快恢复）。

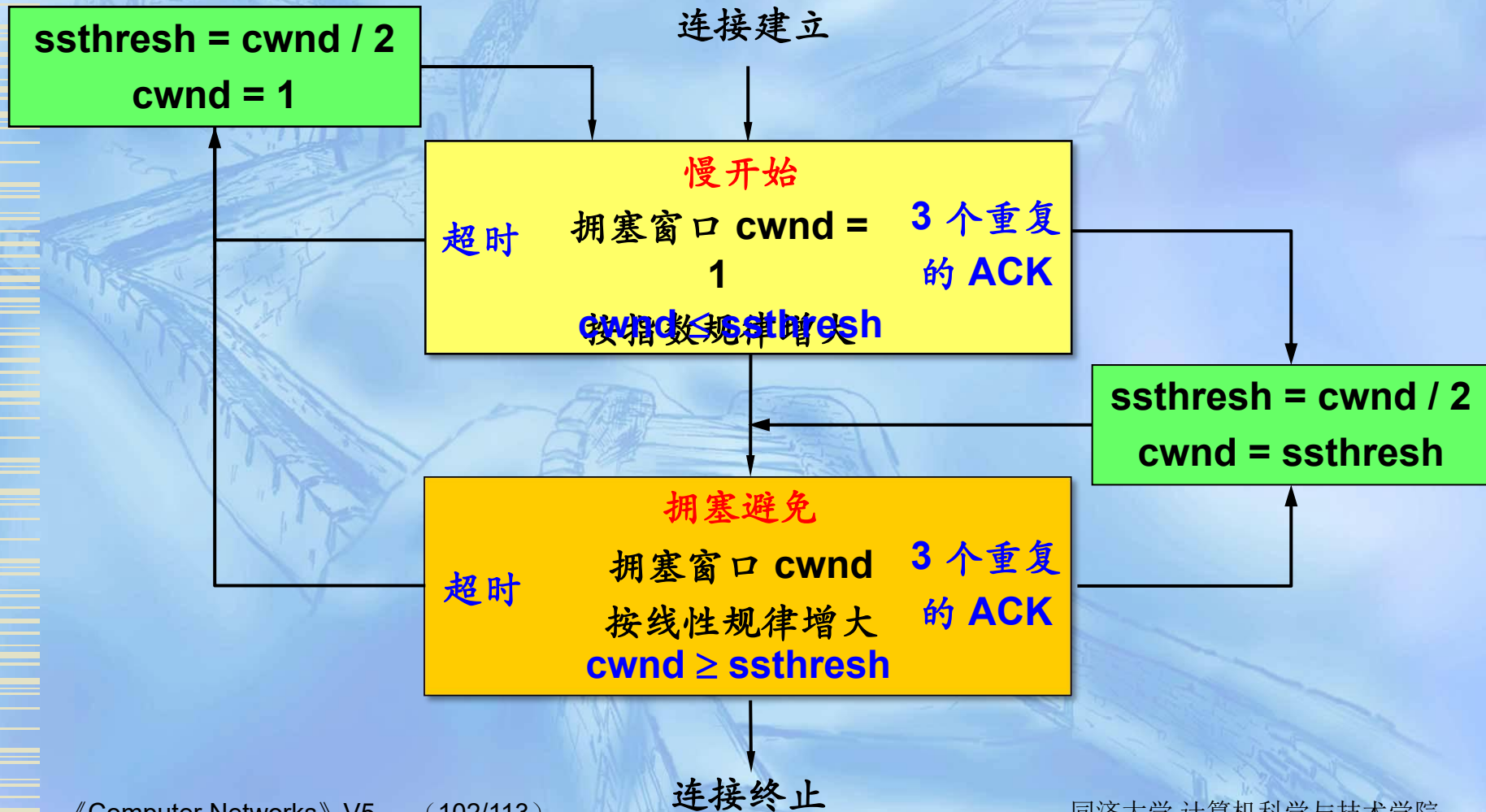


图示





TCP拥塞控制流程图





连接管理

TCP是面向连接的协议。**TCP**通信有三个阶段：**连接建立**、**数据传送**和**连接释放**。

连接管理就是使**TCP**连接的建立和释放都能正常地进行。



建立连接

问题

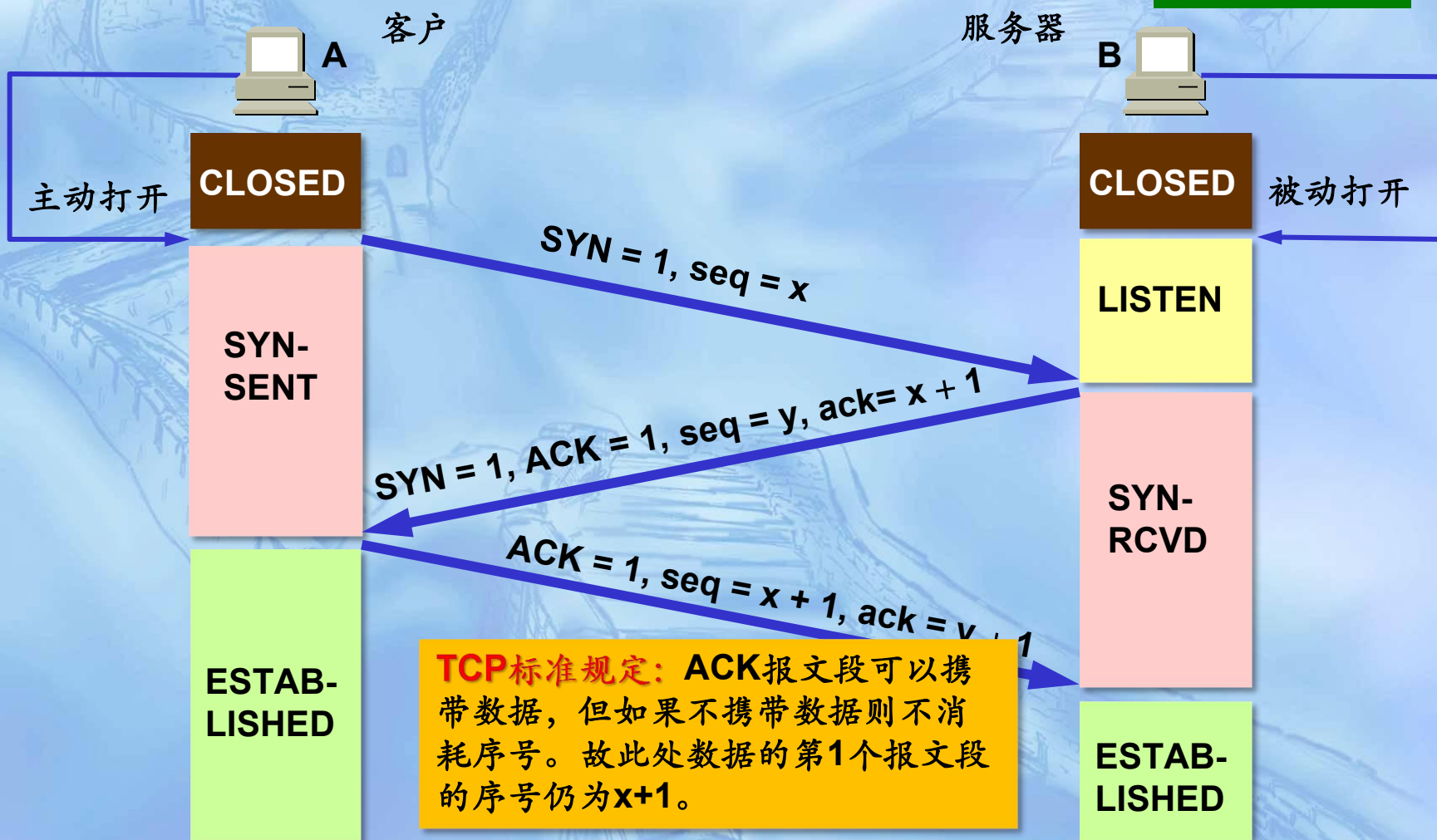
- 要使每一方能够确知对方的存在；
- 要允许双方协商一些参数（如最大报文段长度、最大窗口大小、服务质量等）；
- 能够对传输实体资源（如缓存大小、连接表中的项目等）进行分配。

解决

三次握手。



三次握手建立连接





三次握手具体内容

✎ 一次握手

由客户端发送请求连接报文段，内含要协商的参数：初始序号**ISN**、最大报文段长度**MSS**、窗口扩大因子、**SYN=1**（同步比特）等。

✎ 二次握手

服务器收到请求连接报文段后，如同意，则发送确认（确认号），并内含要协商的参数：初始序号**ISN**、最大报文段长度**MSS**、窗口扩大因子、**SYN=1**（同步比特）等。

✎ 三次握手

客户端收到报文段后，向对方发送确认（确认号）。



释放连接

问题

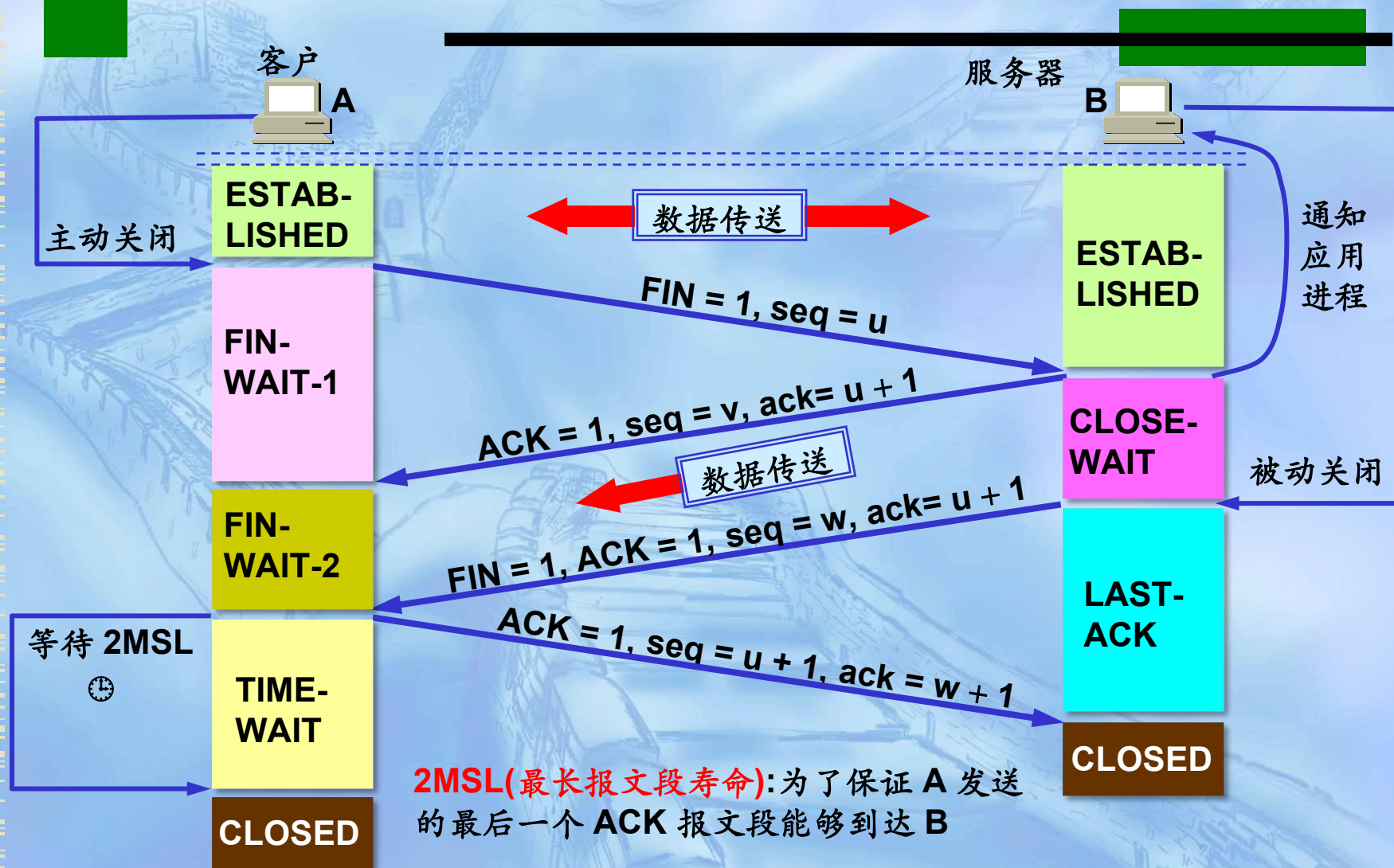
TCP建立的连接是全双工方式。连接的任一方都可以关闭连接，当一个方向的连接被终止时，另外一方还可继续向对方发送数据。如何将两个方向的连接都释放？

解决

四次握手。



四次握手释放连接





四次握手具体内容

✎ 一次握手

客户端发送**FIN**报文段，请求释放连接。

✎ 二次握手

服务器发送**ACK**报文段进行确认，一个方向上的连接释放完毕。

✎ 三次握手

服务器还可以继续向客户端发送数据。如无数据要发送，则发送**FIN**报文段，请求释放连接。

✎ 四次握手

客户端发送**ACK**报文段进行确认，另一个方向上的连接也释放完毕。



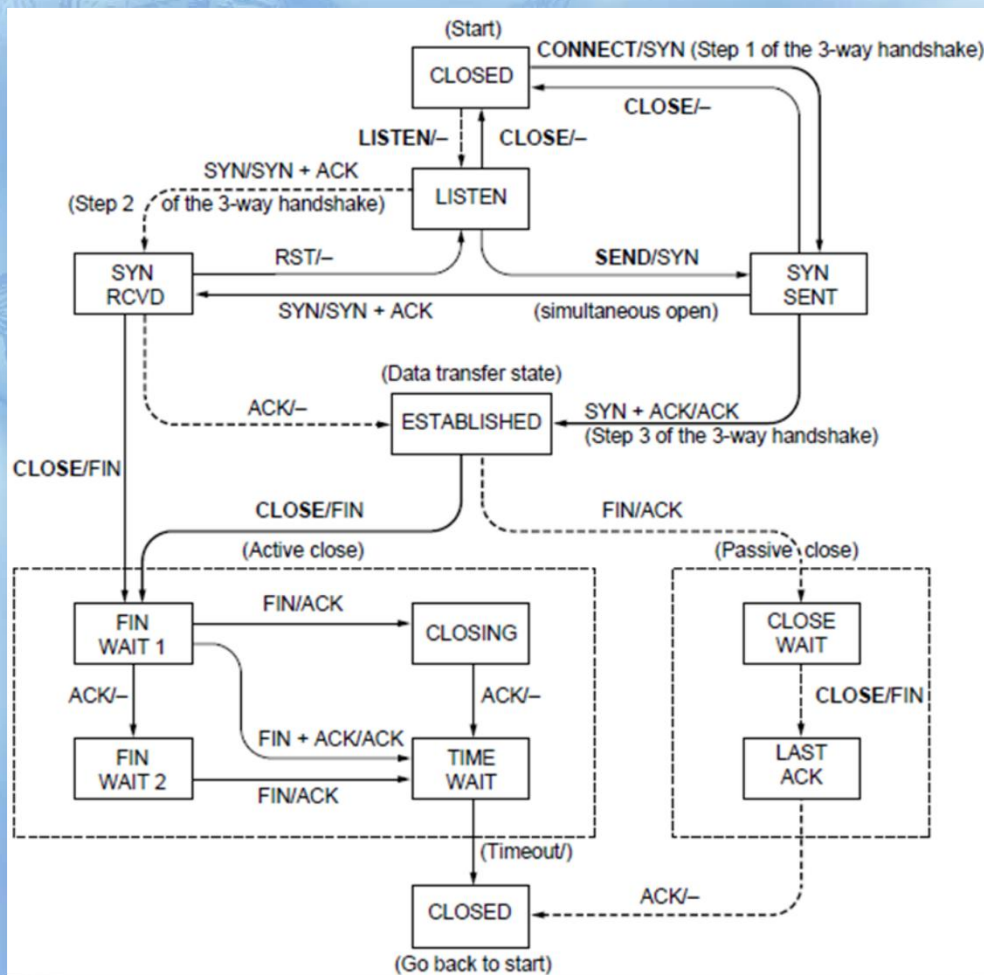
TCP连接管理模型

State	Description
CLOSED	No connection is active or pending
LISTEN	The server is waiting for an incoming call
SYN RCVD	A connection request has arrived; wait for ACK
SYN SENT	The application has started to open a connection
ESTABLISHED	The normal data transfer state
FIN WAIT 1	The application has said it is finished
FIN WAIT 2	The other side has agreed to release
TIME WAIT	Wait for all packets to die off
CLOSING	Both sides have tried to close simultaneously
CLOSE WAIT	The other side has initiated a release
LAST ACK	Wait for all packets to die off

TCP连接管理有限状态机使用的状态



TCP连接管理模型



TCP连接管理有限状态机

粗实线是客户端的正常路径；

粗虚线是服务器端的正常路径；

细线是不常发生的事件；

每次状态变迁由引发的事件以及相应动作标记，事件和动作作用斜杠分割。



性能问题





作业

P467

